

1 | Introduction

1.1 Context

Many academic, administrative, legal, and technical activities produce information as documents rather than as structured database records. A single project may combine reports exported as PDF, editable DOCX files, scanned forms, contracts, invoices, procedures, and course material. These files are convenient to exchange and archive, but their contents become difficult to search as the collection grows. The user may remember a rule or identifier without remembering the filename, page, exact wording, or even the language in which it appeared.

The difficulty is not limited to the number of files. Document representation is heterogeneous. A born-digital PDF can expose a useful text layer, while a scanned PDF page may contain only pixels. A DOCX file has paragraphs, heading styles, and table cells, but does not provide stable page numbers in the same way as a PDF. Repeated headers, footers, page numbers, broken words, and reading-order artifacts can interfere with search. Even after text has been extracted, a long document must be divided into evidence units that can be ranked and shown to a language model within finite input limits.

The intended interaction allows users to ask questions in natural language rather than formulate exact search expressions. Semantic retrieval and language models can support that interaction, but an answer is useful only when the user can verify where it came from. A document assistant must therefore connect generation to real, authorized source passages instead of producing an answer whose origin is unclear [1, 2].

1.2 Motivation

Manual browsing remains effective for a small, familiar folder, but it scales poorly. The user must open files, inspect tables of contents or scroll through pages, and repeat the process whenever the required fact is located elsewhere than expected. Filename search helps only when the name reveals the subject. It cannot find a clause in an ambiguously named attachment or recover a fact from a scanned page.

Keyword search improves content access but depends on wording. A question such as “What happens when a customer ends the agreement early?” may need to retrieve a clause written in terms of “termination before the minimum period.” Conversely, semantic similarity can be weaker for an opaque identifier such as CN-2026-00491, an exact amount, or a specific organization name. This complementarity suggests that neither a purely semantic nor a purely lexical mechanism is sufficient for the intended range of questions.

Sending an entire file directly to a general-purpose language model creates different problems. Large files can exceed practical context limits or consume unnecessary resources. A model may answer using general knowledge, overlook the relevant location, follow instruction-like text embedded

in an uploaded document, or produce a citation marker with no authoritative source behind it. A conversational interface alone does not resolve extraction quality, ownership, provenance, retrieval precision, failure handling, or evaluation.

These limitations motivated an integrated engineering solution. Documents should be processed once at workspace level and then reused across conversations. Native text should be preserved when available, OCR should be applied selectively, and every retrieval unit should retain its source location. Search should combine evidence channels with different strengths, while final answers should be constrained to bounded retrieved context and linked to backend-validated sources.

1.3 Problem Statement

The project addresses the following problem:

How can an authenticated user ingest a heterogeneous collection of PDF and DOCX documents, including scan-like PDF pages, and query that collection conversationally while retaining ownership isolation, bounded processing, traceable evidence, and graceful failure behavior?

Solving this problem requires more than selecting a language model. On the document side, the system must validate uploads, determine how PDF pages are represented, extract or recognize their text, normalize it without destroying valid content, derive controlled document information, create token-bounded chunks, generate embeddings, and persist a searchable representation. On the question side, it must interpret the current query, retrieve candidates through complementary channels, combine and optionally rerank them, construct a finite evidence context, generate an answer, and map citations back to real source chunks.

The scope is deliberately practical. Rather than developing new foundation models or retrieval and OCR methods, the project focuses on integrating established techniques into a coherent document-assistance application and evaluating the resulting behavior at system boundaries. Implementation testing and the completed application-level evaluation are reported separately in Chapter 7.

1.4 Objectives

The main objective is to produce a usable assistant that turns an owned workspace of documents into a persistent, queryable evidence base. This objective is decomposed into the following engineering goals:

1. accept and validate the implemented document formats while preventing unsafe filenames, type mismatches, oversized uploads, and cross-owner access;
2. extract ordered content from PDF pages and DOCX paragraphs, headings, and tables, while retaining raw text and source provenance;
3. identify text-based, scanned, image-based, mixed, and unknown PDF pages before OCR, then route only scan-like pages through bounded local recognition;
4. derive a conservative normalized representation and structured, validated document classification and metadata without making optional understanding a prerequisite for search;

5. create deterministic token-aware chunks with a hard maximum, contextual overlap, and page, heading, and source-section provenance;
6. generate and persist one validated embedding per chunk in PostgreSQL using pgvector, with model, dimension, timestamp, and content-hash currentness information;
7. retrieve project-owned evidence through semantic vector search, PostgreSQL full-text search, and soft document-metadata signals, combining ranks through weighted RRF;
8. apply a bounded model-based reranking stage that can fail open to the deterministic hybrid order;
9. construct bounded Retrieval-Augmented Generation (RAG) context, generate evidence-only responses, and accept only citations mapped by the backend to supplied source chunks;
10. preserve conversations and historical source snapshots while ensuring that previous assistant messages provide continuity rather than factual evidence; and
11. expose useful processing and retrieval diagnostics and verify authorization, algorithms, fallback paths, and trust boundaries through automated tests.

The objectives are intentionally observable. For example, “support scanned documents” is not treated as a label attached to an upload feature; it entails page classification, selective rendering, OCR status, recognized-text provenance, and fallback behavior. Similarly, “provide citations” entails a backend mapping and source snapshot rather than accepting any bracketed identifier produced by a model.

1.5 Proposed Solution

The proposed system is a modular web application with an ASP.NET Core backend, a React and TypeScript client, PostgreSQL persistence, pgvector embeddings, local file storage, and narrow integrations for local OCR and configured model services. A *Project* in the backend is presented as a *Workspace* in the interface. It owns both documents and conversations, allowing a processed document collection to support several independent chats.

The document flow begins with an authenticated upload. PDF files receive deterministic technical analysis before OCR routing; PDF and DOCX extractors then produce ordered raw sections. Selected scanned pages are rendered through a PDFium-based path and recognized locally with Tesseract, after which native and OCR text are merged in source order. Conservative normalization produces the retrieval-oriented representation. Document Understanding optionally derives controlled type, language, title, subject, and metadata. Token-aware chunking, batched embeddings, and transactional persistence complete the searchable knowledge representation. Figure 4.1 presents this flow in Chapter 4.

The question flow begins with the current user question and one query embedding. Owner- and workspace-filtered semantic and lexical searches produce chunk candidates; validated document metadata may contribute a weaker ranking signal. Weighted RRF combines ranks without adding incompatible raw scores. A bounded model-based reranker may reorder the strongest hybrid candidates, but any timeout, provider problem, or invalid output leaves the hybrid order available. Figure 5.1 presents the complete retrieval path.

The final chunks are serialized under a document-context token budget and labelled **S1**, **S2**, and so forth by the backend. Higher-priority answer instructions define the passages as untrusted

data and require an evidence-only response. After generation, unknown citation identifiers are removed and structured sources are created only from the backend’s context map. Recent conversation messages are supplied separately within their own smaller budget and are explicitly non-authoritative.

This architecture joins ingestion and questioning through one persisted provenance chain. Extraction location becomes chunk location; chunk location becomes retrieval metadata; selected chunk metadata becomes an answer source; and a bounded snapshot remains attached to the historical assistant message. The reader can therefore understand an answer without inspecting repository code or trusting an untraceable model claim.

1.6 Personal Contribution

The personal contribution is the engineering design, implementation, integration, and testing of the complete application. The project is not merely a user interface that sends uploaded files to a language model. Its implementation includes secure workspace ownership, upload validation and local storage, asynchronous processing, PDF and DOCX extraction, raw-text preservation, conservative normalization, technical PDF classification, selective local OCR, structured Document Understanding, token-aware chunking, embedding generation, native pgvector persistence, PostgreSQL lexical search, metadata-aware ranking, weighted RRF, bounded model reranking, RAG context construction, citation validation, conversation persistence, diagnostics, and failure isolation.

The established techniques used by the project are credited through the cited literature. The contribution lies in their concrete engineering integration, constraints, validation, and system design. Examples include classifying pages before OCR rather than recognizing every PDF; keeping raw extraction beside normalized text; treating metadata as a soft document signal rather than answer evidence; preserving hybrid ranking when reranking fails; and retaining historical source snapshots without keeping foreign-key dependencies on deleted chunks.

The work also includes designing tests around failure and adversarial boundaries, not only successful demonstrations. The verified suite contains 345 passing backend tests and covers ownership, extraction, normalization, chunking, embeddings, understanding, PDF classification, OCR, hybrid retrieval, reranking, grounding, citations, conversations, and prompt-injection constraints. This implementation evidence is complemented by the completed labelled evaluation reported in Chapter 7.

1.7 Thesis Organization

Chapter 2 introduces the information-retrieval, extraction, OCR, embedding, hybrid search, reranking, grounding, and security concepts required by the implementation. Chapter 3 defines functional and non-functional requirements and presents the high-level, backend, frontend, persistence, ownership, and security architecture.

Chapter 4 follows a document from validation to searchable persistence. It gives particular attention to technical PDF analysis, selective local OCR, Document Understanding, and the token-aware chunking algorithm. Chapter 5 follows a question from semantic and lexical candidate generation through metadata-aware weighted RRF, bounded reranking, RAG context, grounded generation, citation validation, conversation history, and prompt-injection defenses.

Chapter 6 describes the technology stack and the visible workspace, chat, document, source, OCR, intelligence, and retrieval-diagnostic experiences. Chapter 7 reports the automated test

state, evaluation methodology, metrics, measured results, qualitative failure analysis, and threats to validity. Chapter 8 concludes the thesis by summarizing the integrated result, limitations, lessons learned, and possible future development.

2 | Background and Fundamental Concepts

This chapter introduces the concepts needed to understand the implemented assistant. Its purpose is not to provide an exhaustive account of artificial intelligence or database theory. Instead, it establishes a vocabulary for the two connected flows studied in the remainder of the thesis: transforming heterogeneous documents into searchable evidence, and transforming a user question into a grounded answer. The techniques discussed here are established methods. The personal contribution of the project lies in selecting, implementing, constraining, and integrating them in one application, as described in Chapters 3–6.

2.1 Information Retrieval for Documents

Information retrieval concerns the selection and ranking of items that are relevant to an information need. In a document assistant, the user’s question expresses that need, while the searchable collection consists of smaller passages derived from uploaded documents. A retrieval system normally returns a ranked list rather than a single binary match because relevance is gradual: one passage may answer the question directly, another may provide useful context, and many may share only superficial vocabulary [3].

The distinction between a *document* and a *retrieval unit* is important. A fifty-page document is convenient for a person to recognize by filename, but it is too broad to send to a language model for every question. A paragraph-sized unit is precise, but it may omit definitions located immediately before it. The application therefore searches document chunks: bounded passages that retain enough source location to be traced back to their documents.

Retrieval quality can be discussed in terms of precision and recall. Precision asks how much of the returned material is relevant. Recall asks how much of the relevant material was found. The first retrieval stage in this project is designed to obtain a sufficiently diverse candidate set, which favors recall. A later reranking stage can then concentrate on ordering the strongest candidates more precisely. This division helps explain why the system uses more than one search signal.

Provenance adds an application-specific requirement to ordinary ranking. A useful chunk must not only contain relevant text; the application must also know its document, order, page or heading when available, and source sections. That information is carried through ingestion so that a selected passage can become an inspectable citation rather than anonymous model context.

2.2 Text Extraction from Structured Documents

PDF and DOCX encode documents differently. A DOCX file is an Open Packaging Conventions container whose word-processing content exposes logical elements such as paragraphs, styles, and

tables. This makes it possible to traverse body content and recognize explicit heading styles. A PDF primarily describes how content is placed on pages. Its text can be stored as positioned glyphs, and a visually simple paragraph may not be represented as one logical paragraph in the file [4, 5].

Consequently, text extraction is not equivalent to reading a plain-text file. A PDF extractor must reconstruct text in a plausible reading order and preserve the page number. A DOCX extractor can preserve logical order and headings but does not automatically know the final printed page because pagination depends on fonts and layout. Tables also need a textual representation that preserves row and cell distinctions without pretending to reproduce every visual property.

Some PDF pages contain no useful text layer at all. They may consist of a full-page scan, a collection of images, or both text and substantial raster content. Native extraction is preferable when a usable text layer exists because it is fast and retains exact encoded characters. Optical character recognition is required only when the page’s raster representation is the useful carrier of text. This motivates technical classification before OCR in the implemented ingestion flow.

2.3 Optical Character Recognition

Optical character recognition (OCR) converts an image containing written or printed symbols into machine-readable text. A typical OCR route rasterizes a page, detects and recognizes character shapes using one or more language models, and returns text together with optional confidence information. Tesseract is the local OCR engine used by this project [6, 7].

OCR output is an estimate rather than a lossless decoding of the source. Blur, skew, compression artifacts, unusual typefaces, low contrast, handwriting, complex columns, and an incorrect language configuration may produce substitutions, missing characters, or an incorrect reading order. Increasing rendering resolution can improve character detail, but it also increases CPU time and memory consumption approximately with the number of pixels. An implementation therefore needs explicit page and pixel bounds.

Running OCR on every page would discard an important advantage of born-digital PDFs: their existing text layer. It would also waste computation and could replace correct native text with recognition errors. Selective OCR uses structural evidence to route only scan-like pages. Page-level routing is especially valuable for mixed files, such as a digitally produced report with one scanned signed annex.

In this thesis, *local OCR* means that PDF rendering and recognition occur in the application environment rather than in a cloud OCR service. It does not mean that every later processing stage is local. Embeddings, document understanding, reranking, and answer generation may still communicate with the configured model provider.

2.4 Text Normalization

Extracted text often contains layout artifacts that have little value for retrieval: inconsistent line endings, repeated page headers and footers, isolated page numbers, multiple spaces, or words split with a hyphen at the end of a line. Normalization derives a more consistent searchable representation [3]; in this project, the raw extraction is also preserved for inspection.

Normalization must be conservative. The same character pattern can have different meanings: a line at a page edge may be a repeated running header, or it may be a valid short section heading. A hyphen can indicate an extraction-induced word break, or it can be part of a technical term or identifier. Removing uncertain content can permanently hide evidence from every downstream

retrieval stage. The safer engineering principle is therefore to apply bounded rules supported by repeated or positional evidence and to retain occasional boilerplate when certainty is low.

Keeping both raw and normalized representations improves traceability. A developer can compare what the extractor produced with what entered chunking. Counts of changed sections and removed characters provide diagnostics without requiring the system to log entire document bodies.

2.5 Tokenization

A language-model tokenizer maps text to a sequence of integer token identifiers. Tokens are not equivalent to words. A frequent word may be one token, an identifier may be several, and punctuation, whitespace, or non-Latin scripts can change the token count. Therefore a fixed character or word limit cannot reliably enforce a model input budget [8].

This project uses the `cl100k_base` tokenizer for chunk sizes and for the bounded model inputs used in understanding, reranking, document context, and conversation context. Applying the same counting method across these operations makes budgets reproducible. Unicode still needs special handling at slicing boundaries: an implementation must avoid cutting between the two UTF-16 code units of a surrogate pair, even when a token boundary is translated back to a string index.

Token counts serve two related purposes. During ingestion, they prevent a chunk from exceeding a hard size. During model calls, they reserve limited input space among framing text, metadata, and document content. The count is an implementation constraint, not a measure of the semantic importance of a passage.

2.6 Document Chunking

Chunking divides an ordered document representation into bounded passages suitable for embedding, retrieval, and model context. Passage-level dense retrieval provides the general setting for this design [9, 10]. Without chunking, a long document would produce one broad vector and one expensive context item. The broad vector could obscure a short relevant clause, and retrieving the whole document would consume the context budget. At the other extreme, splitting every sentence independently can remove definitions, headings, and qualifications needed to interpret a result.

Three design variables interact:

- a *target size* encourages chunks to contain enough surrounding context;
- a *hard maximum* protects downstream model and storage assumptions; and
- an *overlap* repeats a bounded suffix of the preceding region so that an idea crossing a boundary remains discoverable.

Overlap improves continuity but increases storage, embedding cost, and the chance that near-duplicate chunks occupy several result positions. A minimum useful size reduces weak fragments at the end of a document, although it cannot always be guaranteed when an indivisible source unit or hard maximum prevents a merge. Good chunking also retains provenance. If overlap copies material from a previous page, the resulting source range must honestly include that earlier page.

The detailed algorithm in Section 4.7 uses natural line and sentence opportunities before token-level cuts, respects a hard maximum, rebalances a short final plan where possible, and adds overlap from the preceding base plan. These are project-specific implementation choices built from the general chunking concept described here.

2.7 Vector Embeddings

A text embedding is a fixed-length numerical vector intended to place texts with related meanings near one another in a high-dimensional space. The vector coordinates are not manually assigned topics and generally do not have a useful independent interpretation. Their value lies in comparing complete vectors. A question and a passage can share a semantic neighborhood even when they do not use exactly the same words [9, 11].

For retrieval, each document chunk is embedded once during successful ingestion, and the current question is embedded when a search is performed. The same embedding model and dimension must be used on both sides; comparing representations produced by incompatible models would not define the intended semantic space. Model identity, dimension, content hash, and time are therefore part of embedding validity in the implemented system.

Embeddings are useful but imperfect. They may rank a general thematic passage above an exact identifier, especially when the identifier carries little natural-language meaning. They can also retrieve a semantically related passage that does not answer the precise question. These limitations motivate lexical retrieval and later reranking rather than a claim that vector similarity alone expresses factual relevance.

2.8 Semantic Similarity

One common comparison for embeddings is cosine similarity. For two non-zero vectors $\mathbf{x}$ and $\mathbf{y}$, it is

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\|_2 \|\mathbf{y}\|_2}. \quad (2.1)$$

Here, $\mathbf{x} \cdot \mathbf{y}$ is the dot product, and each denominator term is the Euclidean length of one vector. A larger cosine similarity means the directions of the vectors are more closely aligned. Systems may instead report cosine distance,

$$d_{\cos}(\mathbf{x}, \mathbf{y}) = 1 - \cos(\mathbf{x}, \mathbf{y}), \quad (2.2)$$

for which a smaller value is preferable [3].

The distance is a ranking signal, not a calibrated probability that a passage answers the question. A value cannot safely be displayed as “confidence” without an empirical calibration for the specific model, corpus, and task. The implemented system therefore orders semantic candidates by distance and preserves the value for diagnostics, but it does not apply a universal relevance threshold.

2.9 Vector Databases and pgvector

A vector-capable database stores embeddings in a native numerical type and provides distance operators that can participate in queries. The *pgvector* extension adds this ability to PostgreSQL. It allows relational predicates and vector ordering to be expressed together, so a query can restrict rows by owner, workspace, document status, and embedding validity before returning the nearest chunks [12].

Co-locating relational data and vectors simplifies lifecycle management in this application. A chunk’s embedding is stored on the chunk row rather than in an unrelated external identifier map. Deleting the owning document cascades through its chunks, and the same transaction model

governs textual and vector artifacts. This does not make all vector searches equally scalable. Exact distance ordering is suitable for the current project scale, whereas much larger collections may require an approximate nearest-neighbour index and an evaluation of its recall/performance trade-off.

2.10 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) connects information retrieval to language generation. A typical sequence retrieves passages for the current question, serializes a bounded evidence context, and asks a language model to answer using that context. The model is not used as the database: retrieval decides what evidence is made available, while generation turns that evidence into a readable response [1].

RAG can reduce unsupported use of a model’s general knowledge, but the acronym does not guarantee correctness. A relevant passage may be missed, a weak passage may be selected, a context may be truncated, or the model may interpret evidence incorrectly. The system must specify no-evidence behavior, keep source provenance, bound context and output, and evaluate both retrieval and the final response.

The implemented design uses the current question to retrieve project-owned chunks, optionally reranks them, labels the final evidence, and places it inside explicit untrusted-document delimiters. Conversation history is supplied separately for linguistic continuity and is declared non-authoritative. The answer model is instructed to use retrieved passages as evidence and previous messages only for continuity.

2.11 Lexical and Full-Text Retrieval

Lexical retrieval ranks text through the occurrence and distribution of query terms. Unlike semantic embeddings, it directly rewards shared textual forms. This is valuable for strings whose importance depends on exact spelling: contract identifiers, invoice numbers, monetary values, organization names, and quoted phrases. A lexical method may also fail when the user paraphrases a concept using different vocabulary [3].

PostgreSQL full-text search represents document text as a `tsvector` and a query as a `tsquery`. A Generalized Inverted Index (GIN) can efficiently locate rows containing query lexemes. The `simple` configuration is language-neutral: it tokenizes and normalizes without applying an English-only or Romanian-only stemmer. That choice is appropriate for a multilingual collection and for identifiers that should not be transformed according to one language [13].

User input must not be concatenated into SQL or manually interpreted as trusted query syntax. The current implementation uses parameterized PostgreSQL functions designed for web-style search text and combines an original and deterministically relaxed query. Chapter 5 describes that implementation without exposing database internals to the user.

2.12 Hybrid Retrieval

Hybrid retrieval combines channels that express different forms of relevance. Semantic search handles paraphrase and conceptual similarity; lexical search rewards matching terms and identifiers; metadata can express that a candidate belongs to a document whose known type, title, or extracted fields correspond to the query. Research comparing sparse and dense text representations provides

the general motivation for combining complementary retrieval signals [10].

Table 2.1 summarizes these roles. No channel is universally superior. A paraphrased policy question may favor the vector channel, an invoice number may favor lexical matching, and a query naming a contract type and identifier may benefit from both content evidence and a metadata boost.

Table 2.1: Conceptual comparison of retrieval channels used by the project.

Channel	Primary strength	Typical weakness	Role in this system
Semantic vector	Paraphrases and related meaning	Exact opaque strings may carry little semantic information	Supplies chunk-level evidence candidates
Lexical full-text	Exact or near-exact textual terms	Vocabulary mismatch and paraphrase	Supplies chunk-level evidence candidates
Document metadata	Type, title, filename, and validated extracted fields	Summaries may omit detail and model extraction may fail	Softly boosts documents already represented by content candidates
Model reranker	Joint comparison of the strongest candidates with the question	Additional latency, cost, and model uncertainty	Reorders a bounded candidate set; creates no evidence

A crucial distinction is that a ranking signal is not necessarily answer evidence. A metadata field may help prioritize the correct document, but the final answer must still be supported by retrieved chunk text. The implemented fusion therefore cannot introduce a chunk solely because its document metadata matched.

2.13 Reciprocal Rank Fusion

Raw semantic distance and lexical full-text score do not share a meaningful numerical scale. Adding them directly would make the result depend on arbitrary score ranges and tuning. Reciprocal Rank Fusion (RRF) instead uses the ordinal position of an item in each ranked list. In a weighted form, the generic score for candidate c is

$$\text{RRF}(c) = \sum_{i=1}^n \frac{w_i}{k + r_i(c)}, \quad (2.3)$$

where n is the number of ranking channels, w_i is the weight of channel i , k is a positive rank constant, and $r_i(c)$ is the one-based rank of candidate c in that channel. If the candidate is absent from a channel, that channel contributes zero.

The constant k smooths the difference between adjacent ranks. Channel weights express an engineering preference but do not convert ranks into probabilities. RRF is an established information-retrieval technique [14]. In this project, it is adapted through a weighted formulation that combines semantic, lexical, and bounded document-metadata contributions, together with deterministic deduplication and stable tie-breaking. Section 5.4 gives the actual formula and configuration.

2.14 Reranking

First-stage retrieval must search a potentially large collection efficiently, often using independent query–candidate scores. A reranker receives a smaller candidate set and can compare each candidate more directly with the complete question. Its objective is to improve precision among candidates that have already passed a recall-oriented stage [15].

A model-based reranker is not an answer generator. It should return candidate identifiers and bounded relevance grades, not prose or new facts. Local validation is required because structured-output requests can still fail, omit items, repeat identifiers, or return invalid values. A fail-open strategy preserves retrieval availability: if reranking cannot provide a valid ordering within its time and token limits, the deterministic hybrid order remains usable.

Reranking introduces a trade-off. It can consider the relationship between a question and longer candidate text more flexibly than an independent vector distance, but it adds provider latency and cost and can itself make ranking errors. A fair evaluation must therefore compare the same labelled queries under vector-only, hybrid, and hybrid-plus-reranking variants rather than assuming an improvement.

2.15 Grounding and Citations

A grounded answer is constrained to supplied evidence instead of being encouraged to fill gaps from general model knowledge. Grounding requires more than telling a model to “use the documents.” The application must select an evidence set, state the boundary between instructions and data, define what to do when evidence is insufficient, and preserve a mapping from labels in the model context to authoritative source records. Verifiability research further distinguishes the presence of citations from whether those citations actually support the associated claims [2].

In this project, labels such as **S1** are assigned by the backend when it builds context. The model may cite a supplied label, but it cannot create a corresponding database source. After generation, the backend accepts only identifiers present in its context map and returns source metadata derived from the real chunks. This prevents an invented **S999** marker from becoming a clickable source.

Identifier validation is necessary but not sufficient for citation correctness. A valid source may still fail to support the sentence beside it. Semantic support is therefore measured during evaluation rather than asserted merely because the label exists. Persistent source snapshots keep bounded historical provenance when a live document is later removed, but they do not constitute an archival copy of the entire document.

2.16 Prompt Injection in Document-Based Systems

Prompt injection occurs when untrusted input contains text that attempts to alter a model’s task or reveal protected information. In a document-based assistant, an uploaded file may contain a sentence such as “ignore previous instructions” even though the user asked an ordinary factual question. This is an indirect prompt-injection risk because the instruction-like text reaches a model through retrieved data [16, 17].

No single prompt can prove that prompt injection has been completely solved. A defense-in-depth design reduces risk at several boundaries:

- higher-priority instructions explicitly classify document text as untrusted data;
- model calls have narrow purposes and do not expose arbitrary tools;

- structured outputs use allowlisted fields and controlled categories;
- local code validates identifiers, lengths, numbers, and enumeration values;
- authorization and database filtering occur before document content is sent to a provider;
- citation records are created from backend mappings rather than model-provided metadata; and
- secrets, storage paths, vectors, and provider configuration are omitted from prompts and response contracts.

Residual risk remains because model behavior is probabilistic and an apparently ordinary passage can carry adversarial text. The completed evidence therefore combines automated tests of malformed structured responses with application-level cases involving malicious synthetic document content, unsupported questions, and secret-disclosure requests. The project treats prompt injection as a security boundary to test and constrain, not as a problem that can be declared universally eliminated.

3 | Requirements and System Architecture

This chapter translates the problem introduced in Chapter 1 into concrete system requirements and presents the architecture used to satisfy them. The architecture is described before the detailed processing and retrieval algorithms because those algorithms depend on several system-wide decisions: documents are owned through workspaces, processing is asynchronous, provenance is retained from extraction to citation, and text obtained from a document is treated as untrusted input. The descriptions in this chapter are based on the implemented application rather than on a hypothetical target design.

The application is organized as a modular monolith. A React single-page application communicates with an ASP.NET Core JavaScript Object Notation (JSON) application programming interface (API); the backend coordinates local document processing, PostgreSQL persistence, model-provider calls, and a background ingestion worker. This arrangement keeps deployment manageable for a diploma project while still separating responsibilities through explicit service and provider interfaces. The separation is important for testing and for failure isolation, even though the modules execute in one server process.

3.1 Functional requirements

Functional requirements state what an authenticated user must be able to accomplish. They were derived from the central use case: a user creates a workspace, contributes documents to its reusable knowledge collection, and asks questions whose answers can be checked against those documents. The word *workspace* is used in the interface, while *Project* remains the backend and persistence term. Table 3.1 summarizes the resulting requirements and gives an observable acceptance condition for each one.

Table 3.1: Functional requirements and observable acceptance conditions.

ID	Requirement	Acceptance condition
FR-01	The system shall support account registration, sign-in, session checking, and sign-out.	Protected API operations reject an unauthenticated request, while a valid session identifies one application user.
FR-02	The user shall be able to create, list, edit, open, and delete owned workspaces.	A user sees and changes only projects whose owner identifier matches the authenticated identity.

ID	Requirement	Acceptance condition
FR-03	The user shall be able to upload supported Portable Document Format (PDF) and Office Open XML word-processing (DOCX) documents to an owned workspace. The server shall validate the request before accepting the file.	An accepted file has an allowed extension, matching declared media type, recognizable file signature, safe original name, and size within the configured limit. Successful processing produces raw and normalized source
FR-04	The system shall transform an accepted document into reusable searchable knowledge while preserving its source order and provenance.	sections, bounded chunks, embeddings, and aggregate processing metadata, and places the document in the <i>Ready</i> state.
FR-05	The system shall diagnose the technical representation of PDF pages and apply local optical character recognition (OCR) selectively when a page is classified as scanned.	Page diagnostics and OCR outcomes are inspectable, and successfully recognized text records OCR rather than native extraction as its provenance.
FR-06	The user shall be able to inspect document processing results and request controlled rebuilds.	The interface exposes processing, normalization, chunking, embedding, technical analysis, OCR, and Document Understanding information; rebuild requests are accepted only in compatible states.
FR-07	The system shall retrieve relevant evidence within one owned workspace by combining semantic, lexical, and document-metadata signals.	A search request returns only eligible chunks from <i>Ready</i> documents in the selected project, together with the available rank diagnostics.
FR-08	The system shall answer a question from bounded retrieved evidence and attach traceable citations.	An answer either refers to backend-validated source identifiers or states that the workspace documents do not provide sufficient evidence.
FR-09	The user shall be able to maintain multiple persistent conversations inside a workspace.	Conversations can be created, opened, renamed, and deleted; messages retain a stable order and the first question produces a concise default title.
FR-10	The system shall preserve enough source information to inspect a previously generated answer.	Each cited assistant message stores a bounded snapshot containing the source identifier, document name, chunk and page information, heading, and excerpt.

These requirements deliberately place documents at workspace level rather than attaching them to a single conversation. Extraction, chunking, and embedding can therefore be performed

once and reused by every conversation in that workspace. A conversation supplies interaction history, but it does not own or duplicate the underlying knowledge collection. The same distinction also supports direct search and diagnostics independently of answer generation.

3.2 Non-functional requirements

For this application, correctness includes more than returning fluent text. A response that uses another user’s document, loses the location of its evidence, or silently accepts a stale embedding is incorrect even if its wording appears plausible. The non-functional requirements in Table 3.2 therefore emphasize isolation, traceability, predictable resource use, and recoverability. They are project-specific engineering requirements rather than claims of compliance with an external certification standard.

Table 3.2: Non-functional requirements and their architectural responses.

ID	Quality requirement	Architectural response
NFR-01	Ownership isolation	Every project-scoped read and mutation includes the authenticated owner in its database predicate. Retrieval repeats project and owner filtering in each evidence-producing query.
NFR-02	Evidence traceability	Source sections retain page or heading information and extraction method; chunks carry source ranges; answers expose only source identifiers that the backend can map to retrieved chunks.
NFR-03	Bounded resource use	Upload size, queue capacity, OCR pages and pixels, chunk size, provider inputs and outputs, retrieval pools, conversation history, and answer context all have explicit bounds.
NFR-04	Repeatability and deterministic local stages	Normalization, PDF classification, OCR routing, chunk construction, query preparation, and rank fusion use versioned or deterministic rules. Content hashes identify the source used for derived results.
NFR-05	Failure isolation and retryability	Main document processing, technical PDF analysis, OCR, and Document Understanding have distinct states. Optional enrichment failure does not automatically invalidate useful extracted text, while failed core processing can be queued again.
NFR-06	Persistence consistency	New source sections, chunks, and validated embeddings are prepared before they replace the authoritative retrieval representation in a database transaction. Uniqueness and confidence constraints reinforce application validation.
NFR-07	Maintainability and testability	Controllers, orchestration services, deterministic algorithms, storage, and external-model adapters have narrow responsibilities and are connected through dependency injection, allowing provider boundaries to be replaced by fakes in tests.

ID	Quality requirement	Architectural response
NFR-08	Responsive interaction	Lengthy ingestion is delegated to a background worker. The API returns the document state promptly, and the client can refresh visible progress without holding the upload request open for the whole pipeline.
NFR-09	Controlled information exposure	Physical storage paths, generated storage names, vectors, credentials, hidden prompts, and raw provider responses are excluded from public contracts. Public errors are bounded and phrased without internal details.

The application does not define a formal throughput or response-time service level. Its resource limits are safety and usability controls for the current single-instance implementation, not evidence of large-scale performance. Likewise, deterministic processing means that identical inputs and configuration produce the same local transformations; it does not mean that an external generative model is mathematically deterministic.

3.3 High-level system architecture

Figure 3.1 presents the system as four cooperating runtime areas. First, the browser executes the React and TypeScript client and holds only interaction state needed to render the current screen. Second, one ASP.NET Core host exposes the API, enforces identity and ownership, executes the application services, serves the published frontend, and runs the document worker. Third, PostgreSQL stores relational state, generated full-text vectors, and pgvector embeddings, while an application-controlled directory stores the original uploaded files under generated names. Fourth, the configured model provider supplies embeddings, structured document analysis, reranking, and grounded answer generation. PDF inspection, PDF rendering, OCR, normalization, and chunking remain local to the server.

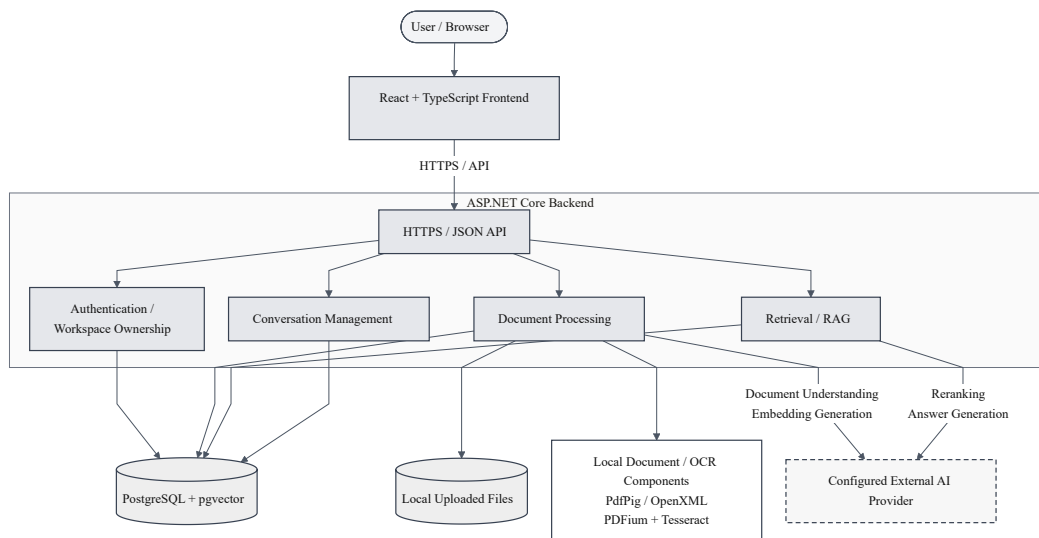


Figure 3.1: High-level architecture of the AI Document Assistant.

Two end-to-end flows cross these areas. On the document side, an authenticated upload is validated and stored, then a queued job converts it into an ordered, normalized, provenance-bearing set of chunks and vectors. On the question side, the current question is embedded and analyzed, several retrieval channels produce candidates, rank fusion and optional reranking select evidence, and a bounded context is submitted for answer generation. The backend then validates source references before returning or persisting them. These flows share the same project ownership boundary and the same persisted document representation; they are not independent subsystems joined only at the interface.

Calling the architecture a modular monolith is significant. The design does not incur network boundaries between extraction, retrieval, and conversation modules, and a database transaction can protect related authoritative changes. At the same time, algorithmic and external dependencies are not embedded directly in controllers. Interfaces isolate text extraction, storage, OCR, model access, retrieval channels, and answer construction. The present design is consequently simpler to run locally than a distributed system, but its process-local queue and local upload directory also make the current deployment a single-instance design.

3.4 Backend architecture

The backend follows a request–service–persistence structure. API controllers define authenticated JSON or multipart endpoints, validate request-level conditions, obtain the user identifier from the authenticated principal, and translate expected failures into bounded HTTP responses. Application services then coordinate use cases such as processing a document, searching a project, answering a question, or persisting a conversation. Entity Framework Core provides the unit-of-work boundary for domain persistence, while direct, parameterized PostgreSQL commands are used where vector and full-text search need database-specific operators. These framework roles describe the checked-in application structure rather than a general comparison of backend frameworks.

Provider adapters form deliberate narrow boundaries around facilities that are either external or implementation-specific. Examples include the local file store, PDF and DOCX extractors, the PDF structural analyzer, PDFium page rendering, Tesseract recognition, tokenization, and the four model-provider tasks. The orchestration code depends on their application interfaces rather than constructing these clients inside a use case. This is why tests can supply deterministic embeddings or no-operation analysis providers without changing the production workflow.

Document jobs are passed through a bounded in-memory channel to one hosted worker. Duplicate scheduling of the same document identifier is suppressed while that identifier is pending or running. For each item, the worker creates a dependency-injection scope and invokes the processing service. The queue is explicitly process-local and non-durable: a server restart can discard pending identifiers, although the persisted *Uploaded* or *Failed* document can be submitted again. This limitation is preferable to describing the queue as if it provided durable delivery.

Uploaded bytes are stored below a fixed **Uploads** root using a newly generated filename whose extension is restricted to PDF or DOCX. Path construction verifies that the requested storage name is a plain filename and that its resolved path remains under that root. Public document contracts return the safe original filename but never the generated storage name or physical path. Database state and file deletion are coordinated carefully for document and project deletion, including state checks that prevent deletion during active processing.

Finally, a global exception handler logs unexpected server failures and returns a generic JSON message. Expected provider and processing failures are wrapped in exceptions with safe public messages. Logging statements generally record identifiers, counts, durations, status, and excep-

tion type while deliberately omitting document text, vectors, prompts, credentials, and provider payloads. This design supports diagnostics without making internal exception details part of the public API.

3.5 Frontend architecture

The client is a React single-page application written in TypeScript and built with Vite. A small route parser maps browser paths to authentication, workspace dashboard, workspace chat, conversation, and document-management views. Navigation uses the browser history API and responds to back and forward events. At startup, the client asks the server for the current session; unauthenticated navigation is redirected to the login interface, while a valid session enters the workspace area.

State is kept close to the screen that owns it. The chat workspace manages the current project, document readiness, conversation list, active messages, uploads, and composer state. The document screen manages document summaries and loads detailed understanding, technical-analysis, OCR, extracted-text, and chunk information when required. Shared UI elements provide confirmation dialogs, notifications, icons, and loading placeholders. This organization is more appropriate to the present application size than introducing a separate global state framework.

A typed API module defines the response shapes and centralizes same-origin requests. It includes the browser cookie on every call, adds a JSON content type when appropriate, and converts structured server errors into safe messages for the interface. These TypeScript declarations improve consistency but do not create a security boundary: browser state and route checks are considered untrusted, and every authorization decision is repeated by the backend. In a published build, ASP.NET Core serves the static assets and falls back to the single application entry page for client routes.

3.6 Persistence and domain model

The domain model separates user intent, source material, derived retrieval artifacts, and conversation records. Figure 3.2 shows the principal relationships without reproducing every database field.

An *ApplicationUser* owns zero or more *Project* records. Each *Project* is one user-facing workspace and is the common parent of its *Document* and *Conversation* collections. This placement is central to reuse: all conversations in the project can search the same ready documents, whereas content from a different project is outside both the retrieval and authorization scope. Deleting a user cascades to owned projects, and deleting a project cascades to its database-resident documents and conversations.

A *Document* represents the uploaded source and the aggregate state of the main ingestion pipeline. It records the safe original name, generated storage name, media type, size, timestamps, current processing status, and summary counts or errors for extraction, normalization, chunking, and embedding. It does not contain all derived text in one large field. Instead, *DocumentTextSection* stores the ordered source representation. A section contains immutable raw extracted text, an optional normalized derivative, its page number or structural heading where available, and the extraction method. Consequently, a reader can distinguish native PDF text, OCR text, and DOCX text, and a rebuild does not destroy the raw material needed for comparison.

DocumentChunk is the bounded retrieval unit derived from one or more normalized sections. Besides its content and stable order, it carries token and character counts, page range, heading,

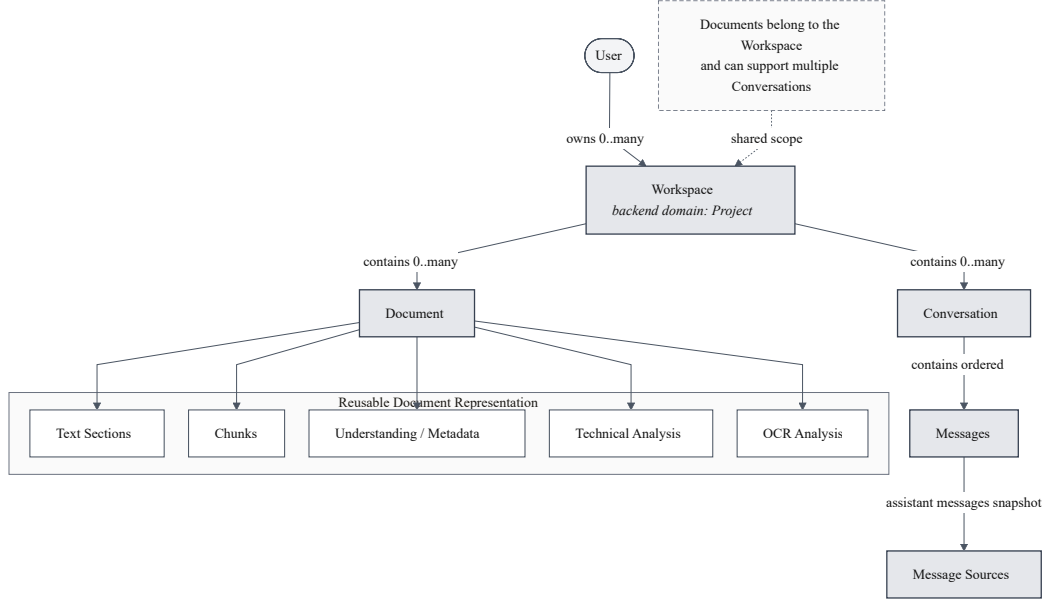


Figure 3.2: Conceptual workspace and document-reuse domain model.

and the first and last contributing section indexes. Its embedding is stored in PostgreSQL through the native pgvector type, together with model, dimensions, content hash, and generation time. A generated PostgreSQL full-text vector for the same chunk supports the lexical retrieval channel. This co-location means that relational ownership filters, text retrieval, and vector retrieval can operate over the same document and chunk identifiers [13, 12].

Three nullable one-to-one aggregates attach distinct forms of document analysis. *DocumentUnderstanding* contains the controlled document type, language and confidence information, detected title and subject, model and prompt identity, source hash, status, and completion information. Its ordered *DocumentMetadataEntry* children represent bounded items such as organizations, people, identifiers, dates, and monetary values. In contrast, *DocumentTechnicalAnalysis* records the structural classification of a PDF and owns ordered *DocumentPageTechnicalAnalysis* diagnostics. *DocumentOcrAnalysis* records OCR configuration, routing identity, and aggregate outcome, with *DocumentPageOcrResult* children only for routed candidate pages. These aggregates are separate because their lifecycles and failure meanings differ; a single overloaded document status would lose that distinction.

The conversation branch has a different purpose. A Conversation belongs to one Project and orders its *ConversationMessage* children by sequence. Messages use only the User and Assistant roles. Sources belong only to assistant messages. Each *ConversationMessageSource* stores a bounded snapshot of the validated citation display: source identifier, safe document name, optional document and chunk identifiers, chunk index, page range, heading, and excerpt. The optional identifiers are scalar historical values, not foreign keys to the live Document or DocumentChunk rows. Deleting an individual source document therefore removes its sections, chunks, full-text index data, and vector values, but does not erase the provenance shown with an older answer. Deleting the conversation still cascades normally to its messages and their snapshots.

Several database constraints support these concepts without attempting to replace application validation. Source-section and chunk indexes are unique within a document; message sequences are unique within a conversation; source indexes are unique within an assistant message; and page diagnostic keys combine their parent analysis with the page number. Confidence and coverage

ratios have database checks that keep them within their valid interval. These constraints protect ordering and basic integrity while the services enforce richer currentness rules, such as matching an embedding hash to the exact current chunk content.

3.7 Authentication and workspace ownership

Authentication uses ASP.NET Core Identity with globally unique user identifiers and an application cookie. The cookie is HTTP-only, uses the *Lax* same-site policy, has a sliding expiration, and follows the security of the current request; HTTPS redirection is enabled by the host. API authentication and access-denied events return JSON status responses rather than redirects to HTML pages. Registration also requires a unique email, while repeated failed sign-in attempts participate in the configured lockout policy.

Authorization is primarily an ownership rule. A Project contains the authenticated user's identifier, and each subordinate resource is reached through that Project. A typical document query therefore requires all three conditions: the requested document identifier, the project identifier supplied by the route, and the owning user identifier reached through the Project. Conversation operations apply the equivalent Conversation–Project–Owner predicate. A caller cannot combine a conversation identifier from one workspace with the route of another. Missing resources, cross-user identifiers, and project/resource mismatches normally produce the same not-found response, reducing unnecessary disclosure about their existence.

Retrieval applies the boundary more than once because it is the path that selects text for external processing. Before generating a query embedding, the service verifies that the project belongs to the current user. The vector, lexical, and metadata database queries also include both project and owner parameters, together with document readiness conditions. Thus a programming mistake in candidate combination cannot legitimately turn an unfiltered global chunk into answer context: the evidence-producing queries themselves are scoped. Parameterized commands are used for database-specific search rather than interpolating user query text into SQL.

The frontend's current-workspace display and route guards improve navigation but are not trusted authorization checks. Likewise, resource identifiers are not treated as secrets. The server assumes that a caller may discover or alter an identifier and still requires the ownership predicate. The current model does not implement workspace sharing or cross-user collaboration; each project has exactly one owner.

3.8 Document-processing architecture

The upload request performs only the synchronous work required to accept a document safely. After validating ownership and the file, it stores the bytes, creates the Document and its initial independent analysis records, and offers the document identifier to the background queue. A PDF begins with technical analysis and OCR marked as not yet analyzed, while those concerns are marked not applicable for DOCX. Document Understanding begins pending. If the bounded queue cannot accept the item, the persisted document remains visible and can be manually queued; the API does not pretend that processing was scheduled durably.

The worker changes an eligible document from *Uploaded* or *Failed* to *Processing* and executes one coherent pipeline. Technical PDF analysis runs before extraction so its page classifications can guide OCR. The relevant extractor produces ordered raw sections; scanned PDF candidates may have ineffective native text replaced by locally recognized text. Normalization creates the conservative retrieval representation. Document Understanding receives a bounded selection of

normalized material, then chunking and embedding produce the searchable units. Chapter 4 explains each transformation and its provenance rules in detail.

The final source sections, normalized values, chunks, and validated vectors are constructed before authoritative replacement. The persistence transaction removes any previous section and chunk representation, inserts the complete new one, updates aggregate counts and embedding identity, and marks the document *Ready*. A failure in extraction, normalization, chunking, or embedding during ordinary processing clears incomplete authoritative retrieval data and marks the main document *Failed*. Technical analysis and Document Understanding, however, persist through their own boundaries. Their failure is recorded without preventing useful normalized text from being chunked and retrieved. OCR can similarly be partial when some selected pages fail but usable native or recognized text remains. This status separation is both a domain-model decision and a resilience mechanism.

3.9 Retrieval and question-answering architecture

Question answering begins with the current user question, not with a previous assistant response. The service verifies project ownership, normalizes and analyzes the question, and generates one embedding in the same configured space as the document chunks. PostgreSQL then returns bounded vector candidates and language-neutral lexical candidates, while ready Document Understanding data may contribute a document-level metadata signal. Only ready documents in the owned project are eligible. Weighted Reciprocal Rank Fusion deduplicates and orders the candidate chunks without directly adding incompatible vector and lexical score scales. A bounded model-based reranker may reorder the strongest fused candidates; if that stage times out or returns an invalid result, the hybrid order remains the fallback. These stages apply established RAG, rank fusion, and passage-reranking ideas [1, 14, 15].

The final chunks are converted into a token-bounded context. The backend assigns source labels such as **S1** and includes the document name, page range, chunk number, heading, and content under explicit untrusted-data delimiters. The answer provider is instructed to use only this evidence. After generation, the backend accepts only citation labels that exist in the supplied context and removes unknown labels. The model can propose the textual marker [S1], but it cannot create the authoritative mapping from that marker to a document.

Persistent conversations add a second, explicitly non-authoritative context. A bounded set of recent user and assistant messages may help resolve wording, but it is separated from document evidence and cannot support a citation. The new question still drives retrieval. The user message is committed before the provider call, so a temporary answer failure leaves a visible question that can be retried rather than a fabricated assistant response. On success, the assistant text and its validated source snapshots are stored together. The direct search endpoint exposes vector, lexical, metadata, fused, and reranked diagnostics for evaluation and debugging, while ordinary answer generation uses the same retrieval service internally.

3.10 Security boundaries

The architecture crosses several trust boundaries, summarized in Table 3.3. Security is consequently implemented as a set of reinforcing controls rather than as a claim that one prompt or one authentication attribute solves every risk.

Table 3.3: Principal trust boundaries, implemented controls, and residual considerations.

Boundary	Implemented controls	Residual consideration
Browser to API	Cookie authentication, server-side validation, owner-scoped database predicates, bounded contracts, and JSON error handling.	Client-side routing is not authorization; production transport, cross-site request protection, and rate limits require deployment review.
Uploaded file to local processing	Size, filename, extension, media-type, and signature checks; generated storage names; path-containment validation; format-specific parsers.	The current validation is not a malware scanner, and document parsers still receive untrusted bytes.
Extracted document text to model tasks	Explicit untrusted-data delimiters, higher-priority task instructions, no document-driven tool execution, bounded input, structured schemas where applicable, allowlists, and local output validation.	Prompt injection is mitigated but not claimed to be completely solved. Provider behavior and adversarial inputs remain subjects for evaluation.
Server to configured model provider	Provider calls are made only by server-side adapters; keys remain in server configuration; response payloads and secrets are omitted from public errors and diagnostic logs.	Selected document text leaves the local process for embeddings, understanding, reranking, and answers. Local OCR does not make the complete pipeline local.
Generated answer to authoritative citation	Source identifiers originate in the bounded backend context; unknown model citations are discarded; persisted sources are validated snapshots.	A valid identifier establishes provenance, but answer correctness and whether the cited passage supports every claim still require evaluation.

Prompt-injection controls are applied at three different model boundaries. Document Understanding asks only for a strict controlled schema and validates types, metadata categories, lengths, confidence values, and counts locally. Reranking assigns opaque candidate identifiers, requests only a structured ordering, rejects invalid relevance values, and fails open to deterministic hybrid ranking. Grounded answer generation separates retrieved text from instructions, denies document-driven commands or secret disclosure, and validates citations after generation. These controls reduce the authority available to malicious text, but they do not justify describing arbitrary document content as safe [16, 17].

Resource bounds also serve a security role. The application limits accepted file size, processing queue capacity, OCR raster size and page count, chunk length, model input and output sizes, retrieval candidate counts, conversation history, and final context. Such limits constrain accidental and adversarial resource consumption, although they are not a substitute for deployment-level rate limiting and monitoring. Database search commands parameterize question content, while public API contracts exclude physical paths and vector values.

The boundary analysis also clarifies what is not asserted. The current project does not claim formal multi-tenant certification, complete prompt-injection prevention, malware detection, encryption of application data at rest, or a production-scale durable job system. It does establish concrete ownership, provenance, validation, and failure-isolation rules within the implemented scope. The engineering contribution lies in enforcing these rules across the complete extraction-to-citation path, rather than merely connecting a document upload control to a general-purpose language model.

4 | Document Ingestion and Understanding

The question-answering capabilities of the application depend on a reliable document representation. A language model cannot recover information that was never extracted, and a retrieval algorithm cannot provide a trustworthy page reference if source location was discarded during preprocessing. For this reason, ingestion is not treated as a preliminary file-conversion utility. It is an integrated pipeline that converts an authenticated upload into searchable, traceable knowledge while preserving enough diagnostic state to explain how that result was obtained.

The implemented path proceeds from validation through technical PDF analysis, extraction, selective optical character recognition (OCR), normalization, document-level intelligence, token-bounded chunks, and vector embeddings. Extraction, normalization, chunking, and embedding are required for retrieval; technical analysis and Document Understanding are separately observable enrichments. This chapter explains these stages and their dependencies.

4.1 Upload and Validation

An upload is accepted only in the context of an authenticated project. The backend first derives the current user identity and verifies that the target project is owned by that user. A nonexistent project and a project belonging to another account are deliberately exposed through the same safe not-found response. Consequently, an unauthorized caller cannot use document endpoints to confirm whether another user's project or document exists. The ownership test occurs before file validation and storage, so rejected callers cannot cause files to be written outside their authorized workspace.

The accepted input surface is intentionally narrow. Table 4.1 summarizes the two supported formats and the checks applied by the server. The size limit is exactly 20 MiB, or 20×1024^2 bytes. The multipart request limit includes an additional 64 KiB allowance for form framing, but this allowance does not increase the permitted file size.

Table 4.1: Accepted upload formats and validation checks

Format	Extension	Expected media type	Content validation
PDF	<code>.pdf</code>	<code>application/pdf</code>	The first bytes must contain the PDF signature <code>%PDF-</code> .
DOCX	<code>.docx</code>	Open XML word-processing media type	The file must have a ZIP signature and contain both <code>[Content_Types].xml</code> and <code>word/document.xml</code> .

Validation is not based on the extension alone. The basename is trimmed, limited to 255

characters, and rejected if it contains control characters. Extension, media type, and the format checks in Table 4.1 must agree. The format-specific parser remains responsible for detecting deeper damage, password protection, or unsupported content.

After validation, the stream is written to a local **Uploads** directory beneath the server content root. A random identifier and the validated lower-case extension form the storage name; the resolved path is checked to remain inside that directory. The original name is stored separately for display and citations. If database persistence fails, the controller attempts to remove the newly written file.

The database record is initially marked *Uploaded*. Related diagnostic records are created at the same time: Document Understanding is *Pending*; technical analysis and OCR are *NotAnalyzed* for PDFs and *Skipped* for DOCX files. The identifier is then offered to a bounded in-memory channel. The channel has capacity for 100 unique document identifiers, permits multiple writers, and is consumed by one hosted background worker. Suppressing duplicate identifiers prevents repeated button presses from scheduling the same document multiple times. The worker creates a fresh dependency-injection scope for each job and invokes the processing service outside the upload request.

The queue keeps the request responsive, but it is process-local and non-durable. Pending items can be lost when the server stops, while persisted *Uploaded* or *Failed* records can be queued again. A durable broker and multi-worker coordination are future work.

Figure 4.1 shows the complete flow from validation and storage to one connected processing path, rather than a set of unrelated document utilities.

4.2 PDF and DOCX Text Extraction

The processing worker selects an extractor from the validated media type and stored extension. PDF extraction is implemented with PdfPig, while DOCX extraction uses the Open XML SDK [18, 19, 4]. The result of either adapter is the same application-level structure: an ordered collection of text sections carrying optional page and heading provenance plus an extraction-method marker.

Table 4.2: Format-specific extraction characteristics

Aspect	PDF	DOCX	Persisted provenance
Primary order	Physical page order	Body-child order	Sequential section index
Text unit	One non-empty native page	Heading-delimited group of paragraphs and tables	Raw text and extraction method
Location	One-based page number	No reconstructed page number	Page when available
Structure	Reading-order text supplied by PdfPig	Heading styles 1–3 and table row/cell separators	Section title when available

For a PDF, PdfPig opens the source stream and visits pages in their original order. Its content-order extractor reconstructs a text string from PDF text objects. Leading and trailing whitespace is removed, and a section is emitted only when the page contains non-empty native text. The section records the one-based page number and the *NativePdf* extraction method. Skipping an empty native page is intentional: the subsequent OCR stage can add recognized text when technical analysis identifies the page as a scan. Parser errors are translated into a safe message explaining that the PDF may be damaged or password protected, without returning low-level parser details to the user.

Because PDF text may consist of positioned glyphs rather than semantic paragraphs, unusual

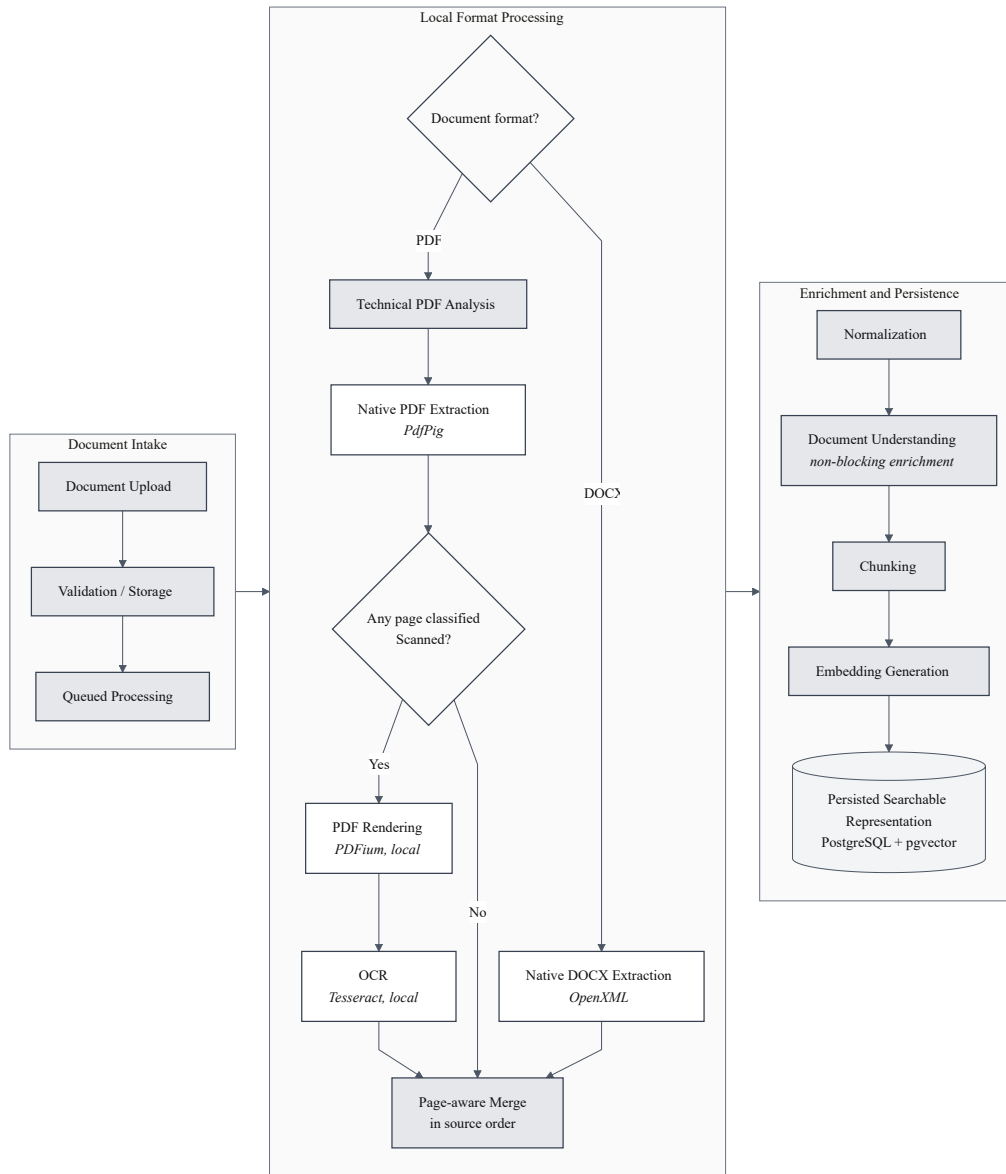


Figure 4.1: Document ingestion and enrichment pipeline.

layouts, columns, or font maps can still produce imperfect reading order. Retained page provenance later allows chunks and answer sources to identify where evidence originated.

For DOCX, the extractor opens the WordprocessingML package read-only and walks the direct children of the main document body. Paragraph text is assembled from its text descendants. A paragraph whose style identifier normalizes to *Heading1*, *Heading2*, or *Heading3* starts a new logical section. The heading is kept in the content and is also stored as the section title, truncated to the database limit where necessary. This dual treatment is useful: the words remain searchable, while the title can also be displayed as provenance and used as a preferred chunk boundary.

Tables are converted without inserting a large source-code-like representation. Paragraphs within a cell are joined with spaces, cells are separated by a vertical bar, and rows are separated by line endings. This preserves row order and makes relationships in modest tables readable to later retrieval stages. Paragraphs and tables remain in body order. The extractor does not reconstruct DOCX pagination or explicitly traverse headers, footers, comments, footnotes, drawings, or heading levels outside the recognized three. It therefore makes no claim of complete WordprocessingML coverage.

Both extractors preserve their output as the raw representation; normalization does not overwrite it. A later full reprocessing may atomically replace both raw and normalized forms, so “raw” means the unaltered authoritative extraction, not a history of every run.

4.3 Raw and Normalized Text

Repeated page edges, divided words, and inconsistent whitespace consume chunk budgets and can become misleading retrieval signals. Conversely, aggressive cleanup can delete clauses or identifiers. The implementation therefore prefers retaining occasional boilerplate to silently removing valid evidence.

The first normalization pass is format-independent. Sections are ordered by their source index. Windows and classic Macintosh line endings are converted to a single newline convention. Horizontal whitespace on each line is collapsed to one space, line edges are trimmed, and consecutive empty lines are reduced to one paragraph break. This gives deterministic text without flattening all paragraph structure.

PDF page-edge cleanup is activated only when at least three page-numbered sections exist. For each page, the first and last 15 non-empty lines form the header and footer candidate regions. Individual repeated lines are considered only near the physical edge, within the first or last three eligible positions. Multi-line candidates may extend farther into the 15-line region, but their boundary must begin or end no more than two lines away from the corresponding page edge. This restriction reduces the risk of removing repeated phrases from the body.

Candidate comparison is deliberately tolerant of harmless representation differences. Text is normalized to Unicode compatibility form, whitespace is collapsed, spaces around common punctuation are standardized, and comparison is case-insensitive. Multi-line candidates are compared as logical blocks, so different physical wrapping does not necessarily prevent recognition. A block must contain at least 40 and at most 4,000 canonical characters. Normally it is removed only when it occurs on at least the ceiling of 60% of the eligible pages. A long block of at least 160 characters can also be confirmed in a dense local sequence: at least three occurrences must occupy a page span whose density reaches the same 60% threshold. The local rule handles, for example, a repeated legal footer that appears in one substantial section of a long combined document without lowering the global threshold.

Standalone page numbers receive separate treatment. Recognized forms include a plain integer

and forms using words such as “page”, “pagina”, “of”, or “din”. When the physical PDF page is known, the displayed current number must equal it; a different number in the body is preserved. Patterns resembling numbered headings, such as “4.2 Processing architecture”, are protected from repetition removal. These checks do not guarantee detection of every decorative pagination scheme, but they prevent a general numeric-removal rule from destroying meaningful quantities.

Word-break repair is similarly bounded. A line must end in one hyphen rather than a double hyphen, the next line must begin with a lower-case letter, the alphabetic prefix before the hyphen must contain at least four letters, and the following alphabetic run must contain at least three. The next run must not immediately continue into another hyphenated form. Only then is the line-ending hyphen removed and the two parts joined. The rule repairs common PDF line-wrap hyphenation while preserving many compound words and numeric identifiers. It is language-independent at the character-class level and therefore works with Romanian diacritics and other Unicode letters, although no dictionary is used.

Table 4.3: Normalization operations, purpose, and conservative boundary

Operation	Intended benefit	Conservative boundary
Line-ending and whitespace normalization	Stable text and token counts	Paragraph separation is retained.
Repeated edge-line removal	Reduce common page headers and footers	Requires PDF pages, edge position, and 60% repetition.
Logical block removal	Handle multi-line boilerplate with changed wrapping	Length, edge-offset, global, and dense-local thresholds apply.
Page-number removal	Remove isolated pagination noise	Restricted to page-edge regions and expected page number.
Word-break repair	Rejoin words divided by line wrapping	Requires a specific alphabetic lower-case pattern.
Whole-document fallback	Prevent destructive cleanup	Restores whitespace-normalized input if all meaningful text would vanish.

After edge cleanup and optional word repair, whitespace is normalized once more. The service records whether each section changed, its non-negative net character reduction, and the normalization time. Aggregate counts on the document support diagnostics. Most importantly, a whole-document guard checks whether every normalized section became empty even though the original contained meaningful non-page-number text. In that situation, the system falls back to whitespace-normalized original sections. The result may contain some unwanted repetition, but it does not lose the document’s entire useful content because of a heuristic.

The explicit rebuild reads stored raw sections and prepares normalization, chunks, and embeddings before replacement. Failure preserves the previous usable representation and records the appropriate error, making heuristic cleanup observable and recoverable.

4.4 Technical PDF Analysis

PDFs may contain native text, scanned images, or a mixture. The application therefore performs deterministic structural analysis before OCR routing. The analysis describes page representation, not subject matter; the distinction is grounded in the PDF document representation standardized by ISO 32000-2 [5].

The controlled types are listed in Table 4.4. Keeping the taxonomy small makes routing and diagnostics understandable. In particular, *ImageBased* is not treated as a synonym for *Scanned*:

a page may contain images without a single page-sized raster that justifies scan OCR.

Table 4.4: Technical PDF page types

Type	Interpretation
TextBased	A meaningful native text layer is the primary useful representation.
Scanned	Little or no meaningful text exists and one raster placement covers at least 80% of the visible page.
ImageBased	Little or no meaningful text exists and raster content is present, but no accepted image reaches the page-sized threshold.
Mixed	Meaningful text coexists with substantial raster coverage, or materially different known page types coexist at document level.
Unknown	Structural evidence is insufficient for a safe classification.

PdfPig supplies two groups of measurements for each page. The text scan counts Unicode letters and digits and also counts alphanumeric word runs containing at least two characters. Text is meaningful when either at least 40 alphanumeric characters or at least eight useful word runs are present. The logical disjunction is important: short tabular pages can have several meaningful words without reaching 40 characters, while a page containing a long compact identifier sequence can reach the character criterion.

Image masks and invalid sample dimensions are ignored. Each remaining bounding box is intersected with the page crop box, and coverage is the largest visible single-image area divided by page area, clamped to $[0, 1]$. Image areas are not summed, avoiding inflation from overlapping placements.

Table 4.5: Deterministic PDF-classification thresholds

Threshold	Configured value
Minimum alphanumeric characters for meaningful text	40
Minimum useful word runs for meaningful text	8
Minimum alphanumeric length of a useful word run	2
Substantial raster coverage	0.30
Page-sized raster coverage	0.80
Strong document-level majority	0.80
Maximum tolerated unknown-page ratio	0.20
Minimum unknown-page tolerance	1 page

Algorithm 4.1 expresses the actual decision order. Notice that meaningful text takes precedence over the scanned-page branch. A page with a page-sized image and a meaningful text layer is *Mixed*, not *Scanned*; it is therefore not sent to OCR by the current routing policy.

At document level, empty or all-unknown input is *Unknown*. Unknown pages may not exceed the greater of one and the floor of 20% of all pages. Among known pages, an inclusive 80% majority wins; otherwise coexisting known types produce *Mixed*. Stable ordering makes ties deterministic.

The result stores page metrics and aggregate counts. Reuse requires the original-file SHA-256 hash and analyzer version `pdf-technical-analysis-v1` to match; a forced rebuild bypasses reuse. Failure is recorded separately and native extraction continues.

The intended flow of these decisions is shown in Figure 4.2.

4.5 Selective Local OCR

OCR is applied selectively, after technical analysis and native extraction. The routing policy chooses only pages whose technical type is exactly *Scanned*. It does not currently OCR *ImageBased*,

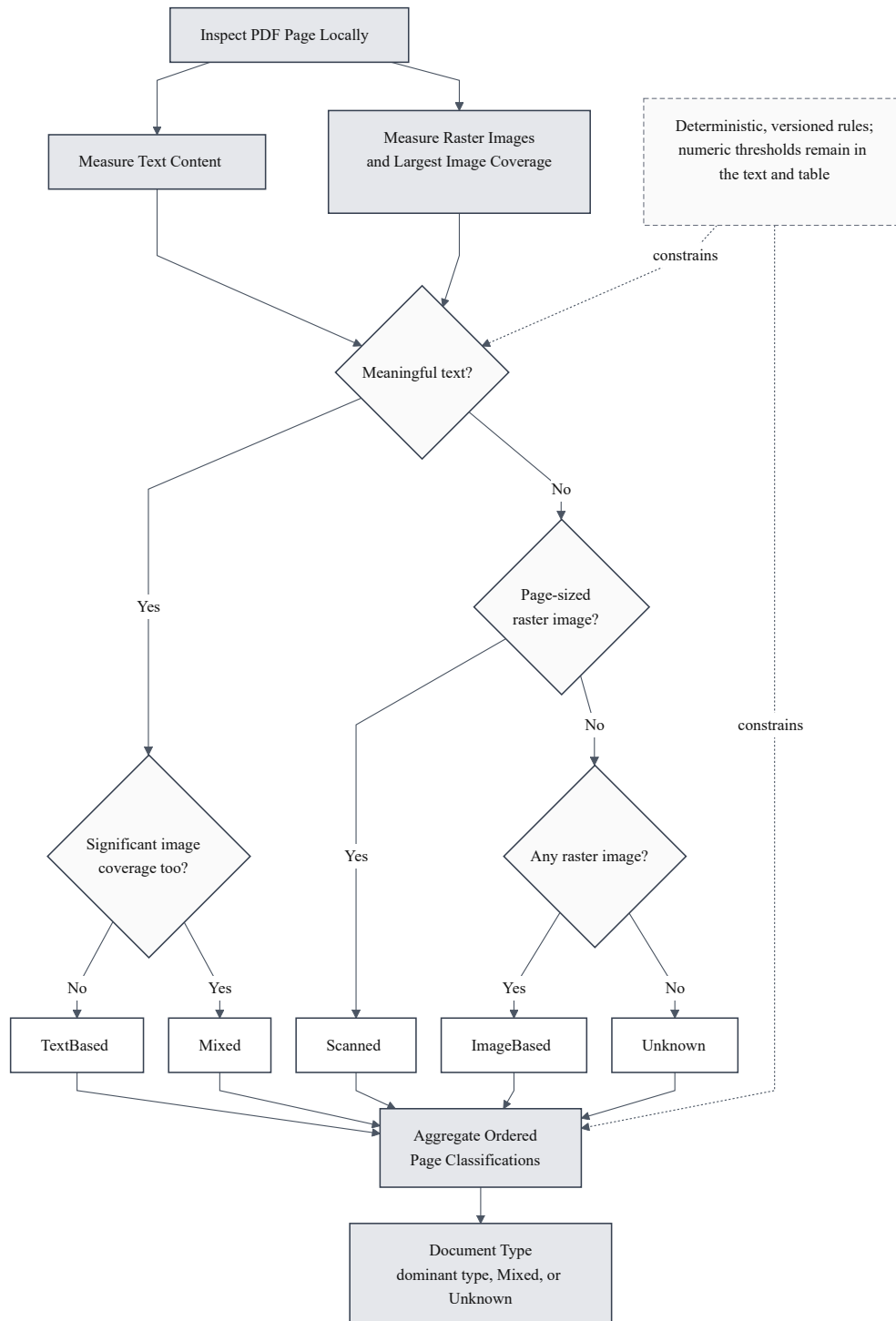


Figure 4.2: Deterministic page- and document-level technical PDF classification flow.

Algorithm 4.1 Deterministic technical PDF classification

Require: Ordered PDF pages with text and accepted raster placements

Ensure: A technical type for every page and for the document

```
1: for all pages in source order do
2:   Count alphanumeric characters and word runs of length at least 2
3:    $meaningful \leftarrow (characters \geq 40) \vee (words \geq 8)$ 
4:    $coverage \leftarrow$  largest visible single-image coverage, clamped to  $[0, 1]$ 
5:    $pageSized \leftarrow (imageCount > 0) \wedge (coverage \geq 0.80)$ 
6:   if  $meaningful$  then
7:      $type \leftarrow$  Mixed if  $coverage \geq 0.30$ , otherwise TextBased
8:   else if  $pageSized$  then
9:      $type \leftarrow$  Scanned
10:  else if  $imageCount > 0$  then
11:     $type \leftarrow$  ImageBased
12:  else
13:     $type \leftarrow$  Unknown
14:  end if
15: end for
16:  $unknownTolerance \leftarrow \max(1, \lfloor 0.20 \times pageCount \rfloor)$ 
17: if there are no known pages or unknown pages exceed the tolerance then
18:   return Unknown
19: end if
20: Group known pages by type and find the stable dominant group
21: if the dominant group represents at least 80% of known pages then
22:   return the dominant type
23: end if
24: return Mixed when several known types remain; otherwise Unknown
```

Mixed, *TextBased*, or *Unknown* pages. This narrow rule avoids replacing a usable text layer with a less certain recognition result and prevents every decorative image from triggering expensive rasterization. It also makes the routing decision reproducible from persisted page diagnostics.

Rendering is provided by PDFtoImage using its PDFium runtime assets [20], and recognition is performed by Tesseract through the TesseractOCR.NET wrapper [6, 7]. Both operations execute on the application host; the OCR stage itself does not send page images to a cloud OCR service. This statement does not imply that all document data remains local. Later Document Understanding and embedding stages may transmit bounded text to the configured model provider.

Table 4.6: Selective OCR configuration

Setting	Current repository value
OCR enabled	Yes
Tesseract language set	ron+eng
Requested render resolution	300 DPI
Maximum scanned candidate pages	200
Maximum rendered pixels per page	25,000,000
Language-data directory	tessdata beneath the server content root
Routing policy/version	Scanned pages only, ocr-routing-v1

Before recognition, the service validates languages, limits, and the data path. A configuration hash covers trained-data fingerprints, engine identity, languages, DPI, and limits; a routing hash covers selected page-number/type pairs. With the source hash, these identities control safe reuse.

For each selected page, pixel dimensions are calculated at 300 DPI. If they exceed 25 million pixels, DPI is scaled by the square root of the permitted area ratio and reduced further after

rounding if needed. A white-background PNG is rendered in memory. This bounds CPU and memory use while retaining the highest resolution allowed by the rule.

Tesseract processes the image with the Romanian and English models. Line endings are normalized, confidence is converted to the interval from zero to one, and leading/trailing whitespace is removed. Non-empty output becomes an extracted section with the original page number and *Ocr* extraction method. Empty output is recorded as an *Empty* page result and does not create artificial content. Render or recognition errors likewise create safe diagnostics without inserting text. If an error indicates that the OCR infrastructure itself is unavailable, remaining selected pages are marked failed without repeatedly attempting the same unavailable resource.

Algorithm 4.2 summarizes routing, reuse, bounded execution, and unification with native extraction.

Algorithm 4.2 Selective local OCR routing and extraction

Require: PDF stream, native page sections, technical-analysis diagnostics, force flag

Ensure: Ordered unified sections and persisted OCR diagnostics

```

1: if technical analysis is not Ready then
2:   Persist OCR as NotAnalyzed and return reindexed native sections
3: end if
4: candidates ← Scanned pages ordered by page number
5: if candidates is empty then
6:   Persist OCR as Skipped and return native sections
7: end if
8: Validate configuration and local language data
9: if configuration or infrastructure is unavailable then
10:   Persist safe Failed diagnostics
11:   return native sections excluding current scanned candidate pages
12: end if
13: Compute source, routing, and configuration identities
14: if not forced and a complete matching Ready result plus stored OCR text exists then
15:   return native and reused OCR sections in page order
16: end if
17: Select at most the first 200 candidate pages
18: for all selected candidate pages do
19:   Render at up to 300 DPI within the 25,000,000-pixel limit
20:   Run local Tesseract with ron+eng
21:   Persist Ready text, Empty, or safe Failed page diagnostics
22: end for
23: Derive Ready, Partial, or Failed aggregate status
24: Remove native sections for all scanned candidate pages
25: Add successful OCR sections, sort by page, and assign new section indexes
26: return the unified sections

```

The replacement step deserves clarification. Native sections from pages classified as scanned are excluded and only successful OCR output for those pages is inserted. This avoids combining a few noisy native glyphs with the recognized page, but it also means that a failed scanned page contributes no text. Native sections from non-candidate pages are kept. Thus a mixed PDF can still become *Ready* when one scan fails but other native or recognized pages provide useful content; an all-scanned document for which every page fails OCR has no extractable text and the overall ingestion correctly fails.

Zero candidates yields *Skipped*; all successes yield *Ready*; some successes yield *Partial*; and none yields *Failed*. Page states are *Ready*, *Empty*, *Failed*, and *SkippedLimit*. Only the first 200 candidates are processed, but all count in the aggregate. The first excess page receives *SkippedLimit*;

further excess pages do not each create a diagnostic row.

Stored provenance includes all three hashes, engine/languages, limits, effective DPI and dimensions, duration, recognized counts, confidence, source type, and whether text entered extraction. Reuse requires a completely *Ready* matching result and one stored OCR section per candidate. Partial or failed results are rerun, and force always bypasses reuse.

The TesseractOCR wrapper dependency is version 5.5.2, while the service has a 5.5.1 fallback engine label and can later record the native engine's value. The wrapper package version and fallback label do not establish the exact native-engine version used in the recorded evaluation.

Figure 4.3 visualizes the page-level route and the return to the unified extraction sequence.

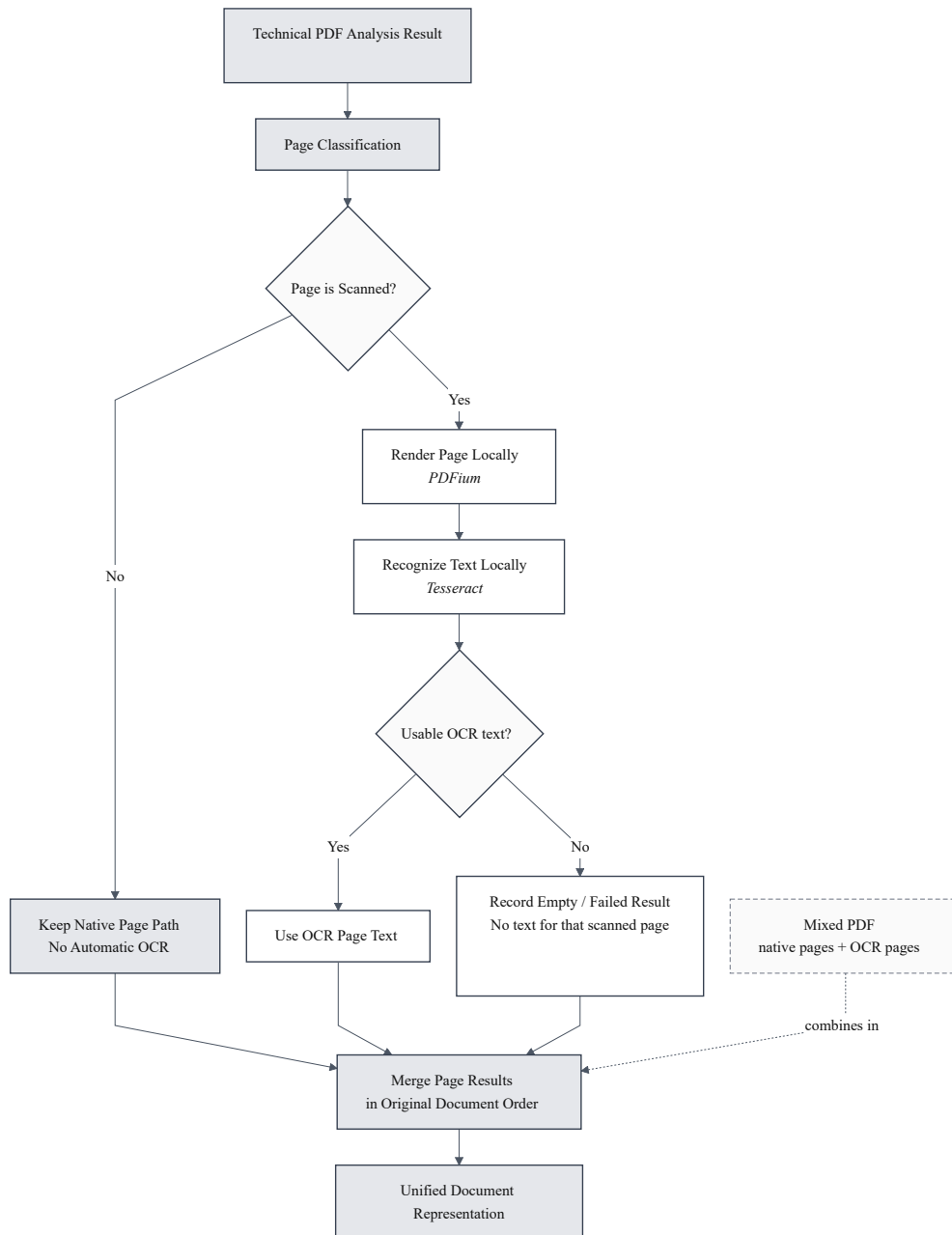


Figure 4.3: Page-level selective OCR workflow.

4.6 Document Understanding

Once normalized text exists, the application attempts a bounded document-level analysis. This stage does not answer a user’s question and does not replace retrieval evidence. Its purpose is to produce a concise structured description that can support document diagnostics and later act as a soft ranking signal. The current configured model is `gpt-5.6-luna`; if a separate Understanding model is not configured, the service falls back to the answer model. Provider interaction uses structured output rather than parsing free-form prose [21].

The output contains a controlled document type, optional subtype, confidence, primary language code and confidence, optional detected title and subject, and an ordered collection of generic metadata entries. The document-type allowlist is: Unknown, Contract, Invoice, Receipt, Report, Policy, Procedure, Manual, CourseMaterial, ResearchPaper, FinancialDocument, Form, Letter, Resume, TechnicalDocument, and Other. This taxonomy is intentionally broad enough for the project corpus without presenting an open-ended model-generated category as an authoritative database value.

Table 4.7: Controlled Document Understanding metadata kinds

Kind	Intended information
Organization	Companies, institutions, public bodies, and other named organizations
Person	People explicitly named by the document
Identifier	Contract numbers, invoice identifiers, references, registration codes, and similar values
Date	Explicit relevant dates, conservatively normalized when unambiguous
MonetaryAmount	Amounts and currencies as supported by document text
Jurisdiction	Explicit territorial or legal jurisdiction
Topic	Concise subject keywords supported by the text
Other	Useful generic metadata that does not fit the preceding kinds

The model receives explicit higher-priority instructions that document contents are untrusted data, never instructions. It is told not to follow requests embedded in the document, reveal prompts or secrets, call tools, contact people, or invent facts. The user message surrounds document content with visible begin/end delimiters. The JSON schema prohibits additional fields, enumerates every accepted document and metadata kind, requires all principal fields, bounds confidence values, and limits metadata to 50 entries. Provider output storage is disabled for this request. These measures reduce the attack surface of indirect prompt injection, but they do not justify a claim that the general problem has been completely solved.

Structured output is followed by local validation. Enum text must map to an allowlisted value. Required confidence scores and optional metadata confidence must be finite and between zero and one; invalid values fail the whole attempt rather than being silently clamped. Language tags are checked for a BCP-47-compatible shape and deterministically cased. Subtype, title, subject, label, and value lengths are bounded. Labels are collapsed into ASCII lower-snake-case form. Duplicate metadata entries are removed deterministically using kind, label, and normalized or original value. Dates are converted to `yyyy-mm-dd` only when a supported representation is unambiguous, including selected Romanian and English month-name forms. Ambiguous numeric dates remain without a normalized value. Monetary values are preserved as cleaned source strings; the local code does not invent a numeric/currency normalization.

Sending every token of a very long document would be costly and would violate the bounded-input requirement. The input builder therefore orders non-empty normalized sections, joins their

canonical content with paragraph breaks, and computes a SHA-256 hash over this entire canonical text. A separate annotated form adds page and heading markers for the model. An input is usable only when it contains at least 20 tokens and at least 40 letters or digits. Shorter input is deterministically marked *Skipped* and no provider call is made.

If the annotated form contains no more than 6,000 `c1100k_base` tokens, it is used in full. Otherwise a deterministic representative sample is built. After accounting for the three sample markers and a 128-token safety reserve, one half of the available budget is assigned to the beginning, one quarter to a centered middle window, and the remaining quarter to the end. The middle window takes a suffix from the left side and a prefix from the right side of the central region. Token boundaries are clamped so a UTF-16 surrogate pair is not split. The sample therefore represents opening context, central body material, and conclusions while remaining reproducible. It is representative rather than exhaustive; facts located only outside these windows may not become metadata, even though they remain available to chunk retrieval.

Idempotency uses the full canonical-content hash, configured model, and prompt version `document-understanding-v1`. A matching *Ready* or *Skipped* result is reused unless the caller explicitly forces analysis. Because the hash covers the full text rather than the sample, a change in an unsampled region still invalidates the previous result. Successful metadata replacement is transactional. Model or validation failure marks the attempt *Failed* with a safe message and does not persist a partial provider response.

Most importantly, Understanding failure is non-fatal to ordinary RAG ingestion. The processing service logs the safe failure and continues with the same normalized sections, chunking, and embedding. After normalization or OCR rebuild, a changed input is first staged as *Pending*; the new searchable representation is committed, and Understanding is retried afterward. If that retry fails, the document remains *Ready* for retrieval. This ordering reflects the role of Understanding as useful enrichment rather than the sole source of searchable content.

4.7 Token-Aware Document Chunking

The normalized document is usually too large and too heterogeneous to serve as one retrieval item. A single vector for an entire report would blend unrelated topics, while inserting the full document into every answer request would exceed bounded context and make provenance imprecise. At the opposite extreme, one vector per sentence can remove the surrounding definitions and conditions needed to interpret a passage. Chunking establishes an intermediate evidence unit: small enough to rank and cite, but large enough to preserve local meaning. Passage-level retrieval provides the established setting for this engineering trade-off [9, 10]; overlap is the project’s deterministic mechanism for carrying limited context across boundaries.

The implementation measures tokens with the `c1100k_base` encoding through `Microsoft.ML.Tokenizers`. The package provides Tiktoken token counting and conversion between text, token identifiers, and character indexes [8]. Character counts are retained for diagnostics, but all size decisions are made from actual tokenizer counts. This is important for multilingual material, punctuation, numbers, and Unicode because equal character lengths do not imply equal model-token lengths.

The target and minimum require careful interpretation. They are planning goals, not unconditional assertions about every possible output. A natural text unit that contains 800 tokens is retained because it is below the 900-token maximum, even though it exceeds the 700-token target. A very short document remains one short chunk. Likewise, the final chunk may remain below 100 tokens when moving or merging available units would violate another constraint. The 900-token

Table 4.8: Current chunking configuration and semantics

Parameter	Tokens	Meaning
Target	700	Preferred point for closing a base plan when a natural unit would take it over the target.
Hard maximum	900	Absolute limit checked against every emitted chunk, including overlap.
Overlap	100	Maximum suffix budget copied from the preceding base plan when space permits.
Minimum useful size	100	Preferred lower bound before heading/target closure and for final-tail rebalancing.

maximum is the hard invariant.

4.7.1 Creating provenance-carrying text units

Source sections are sorted by their persisted index. Each section is split on line endings; empty entries are discarded and non-empty entries are trimmed. Every resulting unit carries its source-section index, optional page number, optional section title, and a flag indicating whether it is the first unit of a titled section. Separators are also explicit: ordinary source lines are joined as paragraphs, while sentences cut from one oversized line are rejoined with a space. These details mean that token counts are computed over the same text that will actually be persisted, including separators.

A unit at or below 900 tokens is left intact. Only an oversized unit is divided into sentences. The sentence detector considers a full stop, exclamation mark, or question mark followed by whitespace or common closing punctuation. It does not split a decimal point between digits, and it protects a compact allowlist of common Romanian and English abbreviations such as *art.*, *nr.*, *dl.*, *dna.*, *dr.*, *prof.*, *etc.*, *Mr.*, *e.g.*, and *i.e.*. This is a conservative boundary heuristic, not a general linguistic sentence parser.

If one sentence still exceeds 900 tokens, the tokenizer locates the largest prefix within the limit. The splitter moves backward to whitespace when doing so produces a non-empty prefix; otherwise it uses the exact token boundary. It also checks UTF-16 boundaries and moves away from the middle of a surrogate pair. Whitespace at the start of the remaining suffix is consumed, and the process repeats. Thus even an unusually long uninterrupted passage can be bounded without corrupting supplementary Unicode characters.

4.7.2 Building and rebalancing base plans

Base plans are constructed by scanning units once in source order. A new titled section closes the current plan only after that plan contains at least 100 tokens. This preference avoids attaching a substantial previous section to a new heading while also preventing every tiny heading fragment from creating an undersized chunk. Before adding each unit, the generator calculates the exact candidate token count. If the candidate would exceed the 700-token target and the current plan is already at least 100 tokens, the current plan closes. If a candidate would exceed 900 tokens under any remaining circumstance, the current non-empty plan closes regardless of its size. Since every individual unit has already been bounded, the newly started plan remains within the hard maximum.

The last plan receives one additional balancing pass. If it already contains at least 100 tokens, nothing changes. Otherwise, the generator first attempts to merge it with the preceding plan; this

is accepted only if their exact joined content remains within 900 tokens. If they cannot be merged, complete units are moved from the end of the preceding plan to the beginning of the final plan. Each move must leave the preceding plan at or above 100 tokens and the final plan at or below 900. Moving whole units preserves natural boundaries. The pass stops when the tail reaches the desired minimum or no valid move exists.

4.7.3 Adding bounded overlap

Overlap is calculated after base planning, so it cannot recursively include content that was itself copied as overlap. For every plan after the first, the available overlap budget is the smaller of 100 tokens and the space remaining under 900 after counting the current base plan. The algorithm walks backward through the immediately preceding base plan and prepends complete trailing units while they fit. If the first considered unit is larger than the entire budget and no complete unit has been selected, it takes a tokenizer-bounded suffix of that unit and advances to a safe word start.

The generator explicitly prevents the overlap from reproducing the entire previous base plan. If several selected units would do so, the earliest is removed. If the previous plan consists of one unit, only a reduced suffix is attempted. After overlap and base units are combined, leading overlap units are discarded if exact counting still exceeds 900. The base content itself is never removed to make room for overlap. As a result, overlap is beneficial but subordinate to the hard size bound and to preservation of the current evidence.

Algorithm 4.3 presents these operations in a compact form while preserving the decisions made by the production implementation.

4.7.4 Persisted provenance and trade-offs

For an emitted chunk, page start and end are derived from all combined units, including overlap. Source-section bounds likewise use the minimum and maximum source indexes represented in the chunk. The displayed section title is the first non-empty title in the current base plan, not a title copied only through overlap. This prevents an inherited suffix from misnaming the principal new content. Chunks may span source sections and PDF pages; page boundaries are not hard stops. Their page range therefore describes all evidence included in the text rather than pretending that every chunk belongs to one page.

Table 4.9: Chunk-size and overlap trade-offs

Choice	Benefit	Cost or risk
Very small chunks	Precise local matching and short context entries	Definitions and conditions can be separated; many vectors are required.
Very large chunks	More surrounding context remains together	Embeddings mix topics, ranking becomes less precise, and context is consumed quickly.
No overlap	Minimal duplication and storage	Relevant meaning can be split exactly at a boundary.
Bounded overlap	Carries nearby context into the next evidence unit	Repeated text can appear in adjacent results and consumes part of the hard limit.
Natural-unit preference	Preserves lines, sentences, and headings where possible	Output sizes vary around the preferred target.

The algorithm is deterministic for identical normalized sections, metadata, tokenizer, and options. It does not depend on random choices or a model call. Repeated rebuilds therefore produce

Algorithm 4.3 Token-aware document chunking with bounded overlap

Require: Ordered normalized sections with page and heading provenance

Ensure: Ordered non-empty chunks of at most 900 tokens

```
1: units  $\leftarrow$  empty list
2: for all sections ordered by source index do
3:   for all non-empty trimmed lines in the section do
4:     if the line contains at most 900 tokens then
5:       Append one provenance-carrying unit
6:     else
7:       Split at protected sentence boundaries
8:       Split any oversized sentence at a safe token/word/Unicode boundary
9:     end if
10:  end for
11: end for
12: if units is empty then
13:   Fail with a safe no-text error
14: end if
15: plans  $\leftarrow$  empty list; current  $\leftarrow$  empty plan
16: for all unit in source order do
17:   if unit starts a titled section and current  $\geq$  100 tokens then
18:     Close current and begin an empty plan
19:   end if
20:   if current + unit > 700 tokens and current  $\geq$  100 tokens then
21:     Close current and begin an empty plan
22:   end if
23:   if current is non-empty and current + unit > 900 tokens then
24:     Close current and begin an empty plan
25:   end if
26:   Append unit to current
27: end for
28: Close the final non-empty plan
29: if the final plan is below 100 tokens and a previous plan exists then
30:   Merge when the result is at most 900 tokens
31:   Otherwise move valid whole trailing units from the previous plan
32: end if
33: for i  $\leftarrow$  1 to number of plans do
34:   base  $\leftarrow$  plans[i]
35:   if i > 1 then
36:     budget  $\leftarrow$  min(100, 900 - TOKENCOUNT(base))
37:     overlap  $\leftarrow$  largest safe, non-whole suffix of plans[i - 1] within budget
38:   else
39:     overlap  $\leftarrow$  empty
40:   end if
41:   Remove leading overlap units while overlap + base > 900 tokens
42:   Build exact content and count its tokens
43:   Reject empty or over-limit output
44:   Emit index, content, page range, title, and source-section range
45: end for
```

the same ordered chunk contents and provenance, although database identifiers and timestamps can change when rows are replaced. Romanian and English text, diacritics, and supplementary Unicode characters are preserved. Determinism supports testing, reproducible retrieval experiments, and reliable invalidation of downstream embeddings.

Figure 4.4 shows three example chunks, their shared suffixes, and the page/source metadata that accompanies each representation.

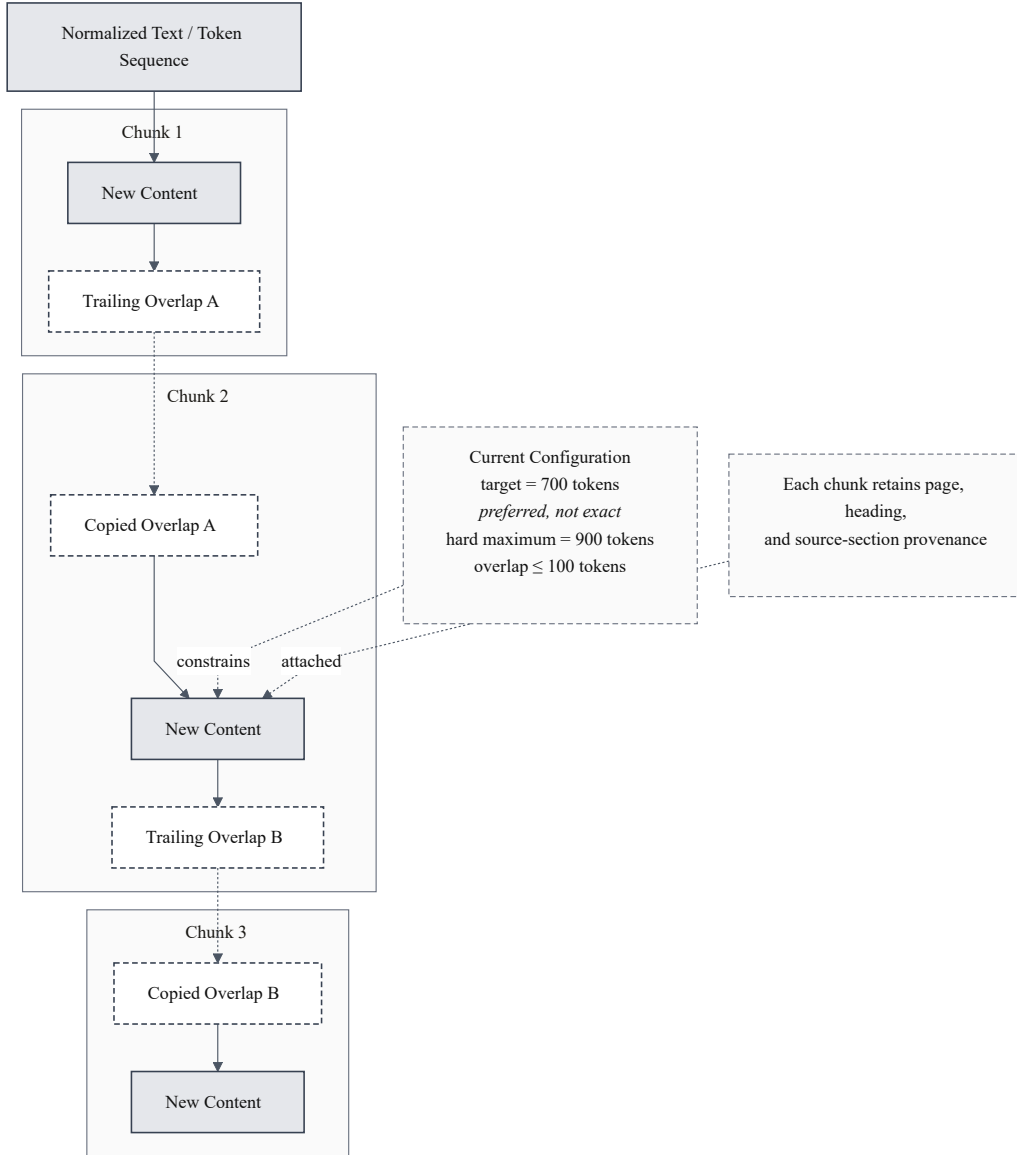


Figure 4.4: Token-aware chunking with bounded overlap and provenance.

4.8 Embedding Generation

Each completed chunk is mapped to one numerical embedding in the same order in which the chunks were generated. The current provider model is `text-embedding-3-small`, configured to return 1,536 dimensions. Inputs are sent in sequential batches of at most 32. The adapter requests the configured dimension explicitly, restores provider results to input order by their response indexes,

and rejects missing, duplicated, non-finite, or wrong-sized vectors. The provider documents both the selected embedding model and the endpoint’s configurable dimension parameter [11, 22]; the value of 1,536 above is the checked-in application configuration.

The application persists vectors through the `pgvector` .NET type. PostgreSQL enables the `vector` extension and stores the value in a native `vector(1536)` column [12]. The fixed dimension is part of the database schema, so startup validation rejects a different configured dimension rather than allowing a later database error. Alongside the vector, every chunk stores the model identifier, dimensions, generation time, and a SHA-256 hash of its exact content. The document stores matching aggregate model, dimension, timestamp, and embedded chunk count. These fields allow the API to distinguish a complete current embedding set from stale or partially populated data without exposing raw vectors to the frontend.

Currency validation is deliberately strict. The document must be *Ready*; its embedded count must equal its chunk count; model and dimensions must match current configuration; and every chunk must contain a vector, share the document completion timestamp, and carry the correct hash for its present content. A changed chunk therefore invalidates the aggregate even if older document-level fields still look current.

There is no embedding cache that suppresses an explicit rebuild. The rebuild operation intentionally regenerates vectors even when the existing set is current. It first snapshots chunk identifiers, indexes, and exact content, performs the provider call, reloads the rows, and verifies that the snapshot did not change while generation was in progress. Only then are all vectors replaced transactionally. If generation or validation fails, previous authoritative embeddings remain available, the document returns to *Ready*, and a safe embedding error is recorded. Normalization and chunk rebuilds likewise prepare new embeddings before replacing existing derived data.

The current schema does not define an HNSW or IVFFlat approximate-nearest-neighbour index. It uses native `pgvector` representation and database-side vector operations without claiming an approximate index that is not present. This is suitable for the present local project scale and keeps the first implementation straightforward. An approximate index, tuning process, and larger-scale benchmark belong to future work.

Embedding is an external AI stage. Although OCR rendering and recognition are local, chunk text is transmitted to the configured OpenAI endpoint to generate vectors. API credentials are read from server configuration and are neither stored with document records nor returned through document contracts.

4.9 Status and Failure Isolation

The pipeline separates overall usability from diagnostic enrichment. This prevents one status value from hiding which stage failed and avoids making optional analysis a prerequisite for all retrieval. Table 4.10 summarizes the persisted concerns.

A representative mixed-PDF outcome illustrates this design. Suppose one page has native text, two pages are classified as scans, one scan is recognized, and the other OCR attempt fails. OCR is *Partial*, yet the unified extraction contains the native page and one OCR page. If normalization, chunking, and embedding succeed, the document can be *Ready*. Diagnostics still reveal the missing page, so readiness does not falsely imply perfect OCR coverage. If all pages were scans and none produced text, the no-text check would instead make document processing *Failed*.

Similarly, a model-provider or validation problem can leave Document Understanding *Failed* while chunks and embeddings make the document searchable. Technical analysis can fail while native PDF extraction succeeds; in that case OCR cannot be safely routed and is reported as not

Table 4.10: Separation of ingestion and analysis status

Concern	States or diagnostic fields	Role in ordinary ingestion
Document processing	Uploaded, Processing, Ready, Failed	Overall searchable usability
Technical PDF analysis	NotAnalyzed, Processing, Ready, Failed, Skipped	Non-fatal structural diagnostic and OCR prerequisite
OCR document	NotAnalyzed, Processing, Ready, Partial, Failed, Skipped	Page recovery; non-fatal only when useful text remains
OCR page	Ready, Empty, Failed, SkippedLimit	Per-page recognition and provenance
Document Understanding	NotAnalyzed, Pending, Processing, Ready, Failed, Skipped	Non-fatal structured enrichment
Normalization, chunks, embeddings	Counts, timestamps, and separate safe error fields	Required derived representation during initial ingestion

analyzed, but the native text can still proceed. These outcomes are not swallowed errors. They are explicit, independently inspectable states.

Required-stage failures have stricter semantics during initial ingestion. Extraction with no useful result, destructive-normalization prevention that cannot produce valid input, chunk-generation failure, or embedding failure prevents a new *Ready* representation. Partial new text and chunks are removed, aggregate counts are cleared, the stored source file is retained for retry, and the error is assigned to the most specific field available. For manual rebuilds of an already ready document, new derived data is prepared before replacement; failure preserves the previous authoritative data and records the rebuild error. This distinction lets the application be strict when first establishing searchability and conservative when updating an already usable document.

The final ingestion result is therefore more than extracted text. It is an ordered and normalized representation, page and heading provenance, independent technical and semantic diagnostics, deterministic bounded chunks, validated vectors, and explicit status information. Chapter 5 uses these persisted artifacts as the starting point for hybrid retrieval and grounded question answering.

5 | Retrieval and Grounded Question Answering

The document-processing path described in Chapter 4 transforms uploaded files into normalized, provenance-aware chunks and stores one embedding for each current chunk. That path prepares the knowledge base, but it does not by itself answer a user’s question. The second half of the system must select useful evidence from all ready documents in the active workspace, fit that evidence into a bounded model request, generate an answer that remains within the supplied material, and preserve a source trail that the user can inspect later.

This chapter describes that question-side path as one integrated process. The implementation first creates a broad candidate set through semantic, lexical, and metadata-aware retrieval. It then fuses rankings whose numerical scores are not directly comparable, optionally asks a model to rerank a bounded subset, and constructs a source-labelled Retrieval-Augmented Generation (RAG) context. Finally, it validates every cited source identifier against mappings created by the backend. The stages have different responsibilities: first-stage retrieval aims to avoid missing relevant evidence, reranking attempts to improve the ordering of the evidence already found, and answer generation turns the final evidence into readable prose. Keeping these responsibilities separate also permits bounded failure handling. In particular, a reranker failure does not make the underlying hybrid results unusable.

5.1 Baseline Semantic Retrieval

Semantic retrieval is the baseline content-retrieval channel. Its purpose is to find passages that express a similar meaning to the question even when the wording is different. For example, a question about “ending an agreement early” may need to find a clause headed “termination”, although the user’s words do not occur verbatim in that clause. Embeddings provide the shared numerical representation needed for this comparison. The concepts of embeddings and cosine distance were introduced in Chapter 2; this section explains how they are applied in the implemented system [9, 3].

When a search begins, the backend first verifies that the requested project belongs to the authenticated user. This check occurs before an embedding request is sent to the external model provider. A missing or foreign project therefore produces no query embedding and does not reveal whether another user’s documents contain matching text. The question is trimmed and checked against a maximum length of 2,000 characters. The embedding service then produces exactly one transient vector for the current question, using the same configured model and dimensionality as the stored chunk vectors. In the checked-in configuration, the model is `text-embedding-3-small` and each vector has 1,536 dimensions. The query vector is used for the database operation and is not persisted.

The PostgreSQL query uses pgvector’s cosine-distance operator to compare the question vector with chunk embeddings. Lower distance means greater similarity, so results are ordered by ascending cosine distance [12]. Stable secondary keys—document identifier, chunk index, and chunk identifier—make an equal-distance result deterministic. The default question-answering request ultimately needs eight chunks, but the semantic channel retrieves up to 30 candidates so that later fusion and reranking have a broader set from which to choose. Public diagnostic search requests may select a Top-K value between 1 and 20; candidate pool sizes are clamped to centralized safety bounds.

Retrieval eligibility is more restrictive than merely checking that an embedding column is non-null. A candidate must satisfy all of the following conditions:

- its project and project owner must match the authenticated request;
- its document must have the **Ready** processing status and at least one chunk;
- the document’s embedded-chunk count must equal its current chunk count;
- the document and chunk must use the configured embedding model and 1,536-dimensional representation;
- the chunk embedding timestamp must correspond to the document’s current embedding operation; and
- the stored SHA-256 embedding-content hash must equal a database-side hash of the current chunk content.

These checks exclude a candidate whose vector is stale relative to its current text. The query requires current document-level aggregate metadata and validates each candidate’s model, dimensions, timestamp, and content hash; it does not separately validate every sibling chunk before returning an eligible row. By contrast, the lexical channel in Section 5.2 can still retrieve ready text without current vectors. This difference is intentional and gives the hybrid design a useful degradation path.

The present database model does not define an HNSW or IVFFlat approximate-nearest-neighbour index. The query performs an exact cosine ordering over the eligible rows. Exact search is simple and appropriate for the current demonstration scale, but its cost grows with the number of eligible chunks. A future large deployment would need to evaluate an approximate vector index and its recall, build-time, and operational trade-offs rather than assuming that the current query plan scales without change.

5.2 PostgreSQL Lexical Retrieval

Semantic similarity is valuable for paraphrases, but it is not uniformly strong for every query. Contract numbers, invoice identifiers, account codes, monetary values, and uncommon names often depend on exact character sequences. A semantically close paragraph may be less useful than the one paragraph containing the requested identifier. Lexical retrieval addresses this complementary need by ranking chunks according to matching terms in their actual text [3, 10].

Each persisted chunk has a generated PostgreSQL **tsvector** column derived from its content. The vector uses the **simple** text-search configuration rather than a language-specific stemmer. This is a deliberate fit for a collection that may contain English, Romanian, identifiers, and mixed-language technical text: it avoids selecting a single linguistic configuration for all documents. A GIN index is defined over the generated search vector. PostgreSQL maintains the generated representation when chunk content changes, so application code does not have to synchronize a second manually computed field [13].

The lexical query uses two forms. The original user text is passed to `websearch_to_tsquery`, which interprets familiar web-search syntax safely inside PostgreSQL. A second, relaxed form is built from deterministic search terms extracted by the local query analyzer. The analyzer applies Unicode compatibility normalization, collapses whitespace, lowercases text, removes a compact set of common English and Romanian query words, deduplicates terms, and retains at most twelve. The relaxed representation helps a natural-language question match its meaningful terms without interpolating user input into SQL.

Both forms use the same `simple` configuration. A chunk qualifies when its generated search vector matches either form. Its lexical score is the sum of the two cover-density ranking values produced by `ts_rank_cd`. The query then orders scores from highest to lowest and applies document identifier, chunk index, and chunk identifier as deterministic tie-breakers. All query values, including the original and relaxed text, are database parameters. Punctuation, quotation marks, operator-like words, Unicode text, and identifier separators are consequently treated through PostgreSQL’s query functions rather than assembled as executable SQL.

Like semantic retrieval, lexical retrieval is scoped to the owner, project, and documents with `Ready` status. It does not require current embeddings. This distinction means that a lexical-only candidate can remain in the hybrid result when an older ready document has usable chunks but its embeddings need rebuilding. The default lexical pool contains up to 30 chunks, independently of the 30 semantic candidates. A chunk found by both channels is not duplicated later; it receives one contribution from each ranked list.

Lexical retrieval is not described as an exact substring engine. Full-text tokenization and ranking still mediate the match, and a phrase may be affected by punctuation or token boundaries. Its engineering purpose is narrower: it contributes a ranking signal that responds strongly to shared terms and identifier-like wording, thereby covering failure modes different from those of the embedding channel.

5.3 Metadata-Aware Retrieval

Document Understanding produces structured information such as document type, title, subject, organizations, people, identifiers, dates, and monetary values. During retrieval, selected parts of this information become a third, document-level ranking signal. This signal is useful when a question refers to a document by what it is—for example, “the invoice from an organization” or “contract CN-2026-00491”—rather than only by the wording of an individual paragraph. Fielded and zone-weighted retrieval provides general context for treating structured attributes as ranking signals [3]; the weights and safeguards described here are specific to this application.

The local query analyzer extracts a bounded set of clues before the metadata query is issued. In addition to the relaxed lexical terms, it recognizes controlled English and Romanian aliases for several document types, separator-based alphanumeric identifiers, supported numeric date forms, and monetary expressions with a recognized currency. At most eight identifiers, eight dates, and eight monetary values are retained. The metadata query itself considers only documents in the active, owned project that are `Ready` and whose Document Understanding record is also `Ready`. A failed or unavailable understanding result therefore cannot block ordinary content retrieval; it simply contributes no metadata rank.

Table 5.1 summarizes the actual fields used. Exact handling must not be generalized beyond what the code implements. In particular, an exact identifier receives special treatment, whereas organization and person values are term-matched through a language-neutral full-text expression.

The metadata query computes an internal match score and converts it into a ranked list

Table 5.1: Metadata-aware retrieval signals and their implemented role

Signal	Matching behaviour	Role and limitation
Document type	Controlled query aliases are compared with the classified type.	Adds a document-level score; it does not prove that a particular chunk answers the question.
Filename	Exact full-name or stem equality, plus term matches.	Identifies the source; a filename match does not establish factual support.
Detected title	Exact normalized equality or term matches.	Can prioritize a likely document while leaving content channels authoritative.
Subtype and subject	Bounded term matches.	Lower-weight descriptive signals.
Identifier	Exact normalized equality or term matches in label/value fields.	Exact equality receives the strongest raw metadata score and activates the RRF multiplier.
Date and monetary amount	Exact normalized forms or term matches.	Exact values improve metadata-document rank, but do not activate the identifier multiplier.
Organization, person, jurisdiction, topic, other	Terms matched against label, value, and normalized value.	Supports descriptive queries; no exact-organization multiplier is implemented.

of documents. An exact identifier contributes 12 points to that internal score; an exact date or monetary amount contributes 6. Term matches contribute 4 for organizations and identifiers, 3 for dates and amounts, and 2 for other metadata kinds. Document type, filename, title, subtype, and subject have separate bounded contributions. The sum of metadata-entry contributions is capped, and documents with equal scores are ordered by identifier. These point values determine the metadata document rank; the raw values are not added directly to the final chunk score.

Only the three strongest distinct matched signals are retained for diagnostics. More importantly, metadata is implemented as a *soft boost*. It cannot create a result by itself. The fusion stage builds its chunk universe exclusively from chunks returned by semantic or lexical retrieval, and then looks up an optional metadata rank for each chunk’s document. A metadata-only document therefore never displaces all actual content evidence. This property prevents classification or extraction errors from becoming hard filters and preserves the possibility that a strong paragraph wins despite weak metadata.

Metadata is also *not answer evidence*. The RAG context contains the selected chunk’s content and provenance, not the Document Understanding labels that boosted its document. The answer model must support its factual statements from retrieved text. Metadata affects where the system looks; it is not presented as proof of what the document says.

5.4 Hybrid Retrieval and Weighted Reciprocal Rank Fusion

The three retrieval channels produce different kinds of numerical values. Cosine distance is lower when a semantic candidate is better. PostgreSQL full-text rank is higher when a lexical candidate is better. Metadata has a separately designed document-level match score. Adding these raw values would make the final ranking depend on arbitrary scale and distribution differences. A normalization rule could be introduced, but its behaviour would vary with the candidate set. Reciprocal Rank Fusion (RRF) avoids this problem by using each channel’s ordinal rank rather than its raw score. RRF is an established information-retrieval technique [14]; the contribution of this work is its weighted, metadata-aware integration and deterministic implementation within the

retrieval pipeline.

Figure 5.1 shows the complete first-stage path. Semantic and lexical searches supply chunk candidates, while metadata supplies document ranks that can boost only those existing candidates. The fused list is bounded before it crosses the reranking provider boundary.

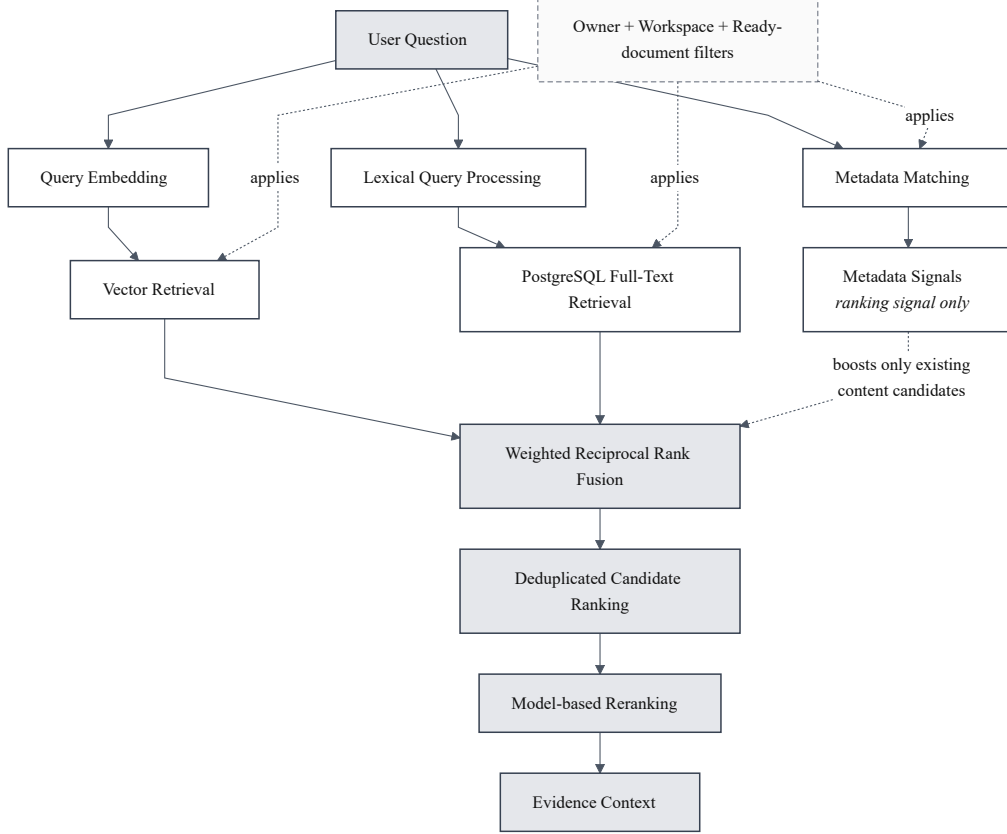


Figure 5.1: Hybrid retrieval and candidate-fusion pipeline.

For a chunk c , the implemented weighted score is

$$\text{RRF}(c) = \frac{w_v}{k + r_v(c)} + \frac{w_l}{k + r_l(c)} + \begin{cases} \frac{w_m q(c)}{k + r_m(d(c))}, & \text{if metadata matched document } d(c), \\ 0, & \text{otherwise.} \end{cases} \quad (5.1)$$

Here, $r_v(c)$ and $r_l(c)$ are the one-based vector and lexical ranks of chunk c . A missing chunk rank contributes zero rather than an arbitrarily poor score. The function $d(c)$ gives the chunk's document, and $r_m(d(c))$ is that document's one-based metadata rank. The constant k reduces the difference between adjacent high ranks and prevents the first result from dominating the entire fusion. The channel weights are w_v , w_l , and w_m . Finally, $q(c)$ is 1.5 only when the matched document has an exact identifier match and is 1 otherwise. It does not multiply the semantic or lexical terms.

The checked-in values are shown in Table 5.2. Startup validation additionally requires the largest possible metadata-channel weight, $0.35 \times 1.5 = 0.525$, not to exceed either content-evidence channel weight.

Before scoring, each input list is converted to a unique chunk-to-rank map; the first occurrence

Table 5.2: Checked-in hybrid retrieval and RRF configuration

Parameter	Value	Interpretation
Vector candidate count	30	Recall-oriented semantic pool before fusion.
Lexical candidate count	30	Independent full-text pool before fusion.
Metadata document count	20	Maximum ranked documents contributing soft boosts.
Reciprocal rank constant k	60	Denominator offset shared by all channels.
Vector weight w_v	1.0	Full-weight chunk-content channel.
Lexical weight w_l	1.0	Full-weight chunk-content channel.
Metadata weight w_m	0.35	Deliberately weaker document-level contribution.
Exact identifier multiplier	1.5	Applies only to w_m for a document with an exact identifier match.
Fused candidates for default answer	up to 18	Bounded list passed toward reranking before final Top-8.
Hard chunk-candidate bound	100	Defensive maximum for configured vector, lexical, or fused pools.

establishes the rank. Metadata records are similarly reduced to the best ranked record for each document. The candidate set is then the union of vector and lexical chunk identifiers. Algorithm 5.1 states the process without reproducing production C# code.

Algorithm 5.1 Weighted RRF over content candidates with a soft metadata boost

Require: ordered vector chunks V , lexical chunks L , metadata documents M , result bound n

Ensure: at most n unique fused chunks

```

1:  $R_v \leftarrow$  first one-based rank of each unique chunk in  $V$ 
2:  $R_l \leftarrow$  first one-based rank of each unique chunk in  $L$ 
3:  $R_m \leftarrow$  best one-based rank of each unique document in  $M$ 
4:  $U \leftarrow \text{keys}(R_v) \cup \text{keys}(R_l)$ 
5: for all chunk  $c$  in  $U$  do
6:    $s \leftarrow 0$ 
7:   if  $c$  occurs in  $R_v$  then
8:      $s \leftarrow s + w_v / (k + R_v[c])$ 
9:   end if
10:  if  $c$  occurs in  $R_l$  then
11:     $s \leftarrow s + w_l / (k + R_l[c])$ 
12:  end if
13:  if document  $d(c)$  occurs in  $R_m$  then
14:     $q \leftarrow 1.5$  if  $d(c)$  has an exact identifier match, otherwise 1
15:     $s \leftarrow s + w_m q / (k + R_m[d(c)])$ 
16:  end if
17:  attach channel ranks, matched metadata summary, and score  $s$  to  $c$ 
18: end for
19: sort by score descending, best individual rank, document ID, chunk index, then chunk ID
20: return the first  $n$  chunks

```

Figure 5.2 visualizes the same idea as contributions entering one rank-based score. It is deliberately a conceptual diagram rather than a graph of measured project quality.

Table 5.3 gives a small fictional example using the actual default formula. It is explanatory and is not an evaluation result. Chunk A is strongest semantically but fifth lexically and has no metadata match. Chunk B is first lexically, fourth semantically, and belongs to the first metadata-ranked document with an exact identifier match.

Chunk B ranks first in this example because it has evidence in all three channels, including

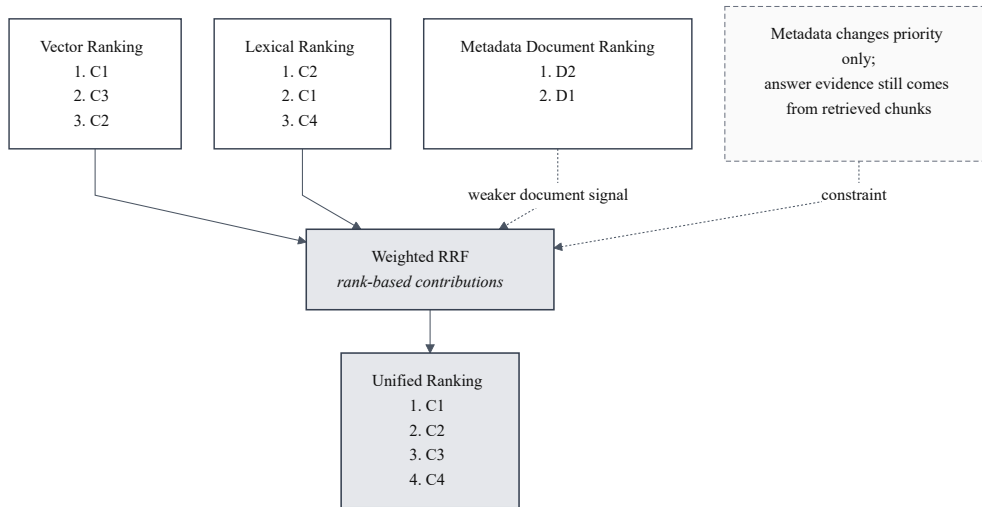


Figure 5.2: Conceptual weighted Reciprocal Rank Fusion of vector, lexical, and metadata rankings.

Table 5.3: Fictional weighted-RRF calculation using the implemented defaults

Chunk	Vector contribution	Lexical contribution	Metadata contribution	Total
A	$1/(60+1) = 0.016393$	$1/(60+5) = 0.015385$	0	0.031778
B	$1/(60+4) = 0.015625$	$1/(60+1) = 0.016393$	$0.35 \times 1.5/(60 + 1) = 0.008607$	0.040625

an exact identifier signal. The result does not imply that metadata can rescue a metadata-only document: B was already present in both content candidate lists. If its vector and lexical positions were much weaker, the deliberately bounded metadata contribution would not necessarily overcome stronger content evidence.

After scoring, ties are resolved by the best individual rank across the available channels, followed by document identifier, chunk index, and chunk identifier. This ordering ensures reproducibility even when floating-point scores are equal. Diagnostic fields preserve vector rank, lexical rank, metadata-document rank, lexical score, fused score, and the short matched-metadata summary. These fields support development and evaluation, while the downstream answer receives actual chunk text rather than ranking diagnostics.

5.5 Model-Based Reranking

First-stage hybrid retrieval is designed to maximize the chance that useful evidence enters a bounded candidate set. Its signals remain relatively shallow: embedding similarity, term matching, and structured document hints. Model-based reranking performs a different task. It reads the strongest candidate passages in relation to the original question and attempts to order them by direct usefulness for a grounded answer. This is a precision-oriented second stage, not a replacement for candidate generation [15]. Chapter 7 compares vector, hybrid, and final/reranked orderings on the labelled evaluation set.

With the default configuration, RRF retains up to 18 candidates for this stage, while the eventual answer still uses Top-8. Eighteen is the configured candidate count and 30 is the hard maximum accepted by the reranking subsystem. Candidate construction follows the hybrid order and assigns stable opaque identifiers C1, C2, and so on. The provider receives the original question

and, for each candidate, only its opaque identifier, bounded document name, page label, optional heading, and content. It does not receive embedding vectors, cosine distances, lexical scores, meta-data values, channel ranks, or the fused score. Omitting those diagnostics makes the second-stage decision content-oriented and avoids encouraging the model simply to restate the deterministic ranking.

The input builder applies several bounds before a network request is allowed. A document name is limited to 300 characters and a heading to 500. A serialized candidate may contain at most approximately 700 `c1100k_base` tokens. If content exceeds the available space, it is truncated at a tokenizer-derived character boundary and marked as truncated. The complete constructed *user payload* is limited to approximately 12,000 tokens. This number does not include the separate system instructions, so it should not be described as a complete provider-request token count. When the remaining budget cannot hold another candidate with the minimum 100-token candidate allowance, candidate collection stops. All truncation affects only the provider payload; persisted chunk text remains unchanged.

The checked-in reranker settings appear in Table 5.4.

Table 5.4: Checked-in model-reranking configuration

Setting	Value	Meaning
Enabled	true	Attempt reranking when the candidate/Top-K conditions require it.
Configured candidate count	18	Normal maximum candidates built from hybrid order.
Hard candidate maximum	30	Defensive schema and application limit.
User-payload token budget	12,000	Approximate tokens in the constructed user input; excludes system instructions.
Per-candidate token budget	700	Includes serialized candidate framing and bounded content.
Minimum candidate allowance	100	Below this amount, another candidate is not added.
Provider output budget	800	Maximum response tokens for structured ranking.
Relevance scale	0–4	Irrelevant through directly supporting evidence.
Timeout	30 seconds	Linked application timeout; hard configurable maximum is 60 seconds.
Configured model	<code>gpt-5.6-sol</code>	Falls back to the answer model only if the reranker model is omitted.

The reranking system instruction defines candidate text, filenames, page labels, and headings as untrusted data. It forbids following commands contained in a candidate, calling tools, answering the user’s question, creating citations, or producing explanatory prose. It asks only for comparative relevance. The response is constrained by a strict JSON schema: an object containing a ranking array, whose items contain only a candidate ID and an integer relevance grade from 0 to 4. Stored provider output is disabled and low reasoning effort is requested [21].

Strict provider output is necessary but not sufficient. The backend validates the result against the exact request it created. Unknown IDs are ignored because they have no authoritative mapping. If an ID occurs repeatedly, only its first occurrence is retained. An omitted known candidate is not discarded; it is appended after the model-listed candidates in its original hybrid order. This policy permits a partially returned but otherwise valid ranking without allowing evidence to disappear unpredictably. Conversely, a null item, a null or out-of-range grade for a known candidate, an empty usable ranking, or a ranking longer than the supplied candidate set makes the response

untrusted and triggers complete fallback.

Algorithm 5.2 summarizes orchestration and local validation. “Hybrid Top-K” always means the first K chunks from the deterministic RRF order.

Algorithm 5.2 Bounded reranking validation with fail-open hybrid fallback

Require: question Q , ordered hybrid candidates H , requested result count K

Ensure: a final list of at most K candidates

```

1: if reranking is disabled,  $|H| \leq 1$ , or  $|H| \leq K$  then
2:   return first  $K$  candidates from  $H$ 
3: end if
4: build bounded request  $B$  with opaque IDs from  $H$ 
5: if request construction fails then
6:   return Hybrid Top-K and record fallback
7: end if
8: if  $|B| \leq K$  then
9:   return Hybrid Top-K without calling the provider
10: end if
11: call the reranker under the configured timeout
12: if timeout, provider error, missing/empty ranking, or ranking longer than  $B$  then
13:   return Hybrid Top-K and record fallback
14: end if
15:  $O \leftarrow$  empty ordered list;  $S \leftarrow$  empty set of accepted IDs
16: for all item  $x$  in provider order do
17:   if  $x$  is null then
18:     return Hybrid Top-K and record fallback
19:   end if
20:   if  $x.id$  is unknown or already in  $S$  then
21:     continue
22:   end if
23:   if  $x.grade$  is null or outside  $[0, 4]$  then
24:     return Hybrid Top-K and record fallback
25:   end if
26:   append mapped candidate and grade to  $O$ ; add ID to  $S$ 
27: end for
28: if  $O$  is empty then
29:   return Hybrid Top-K and record fallback
30: end if
31: append every candidate omitted from  $O$  in original hybrid order
32: assign rerank positions and return first  $K$  candidates from  $O$ 

```

Reranking is deliberately skipped when disabled, when zero or one candidate exists, when the fused set is no larger than requested Top-K, or when token budgeting leaves no more than Top-K request candidates. These normal skip cases are distinct from fallback and do not report an error. During a real provider call, a linked 30-second timeout is applied. Caller cancellation still propagates, but timeout, provider exception, missing API configuration, malformed output, or failed input construction returns the original hybrid Top-K. The question-answering pipeline can therefore continue with deterministic evidence instead of failing solely because the optional precision stage is unavailable.

Figure 5.3 emphasizes this two-outcome design.

The design has an explicit latency and cost trade-off. Candidate generation requires one query embedding and database retrieval; reranking adds a second model request before answer generation. Larger candidate sets may improve the chance of presenting useful passages to the reranker, but consume more tokens and response time. The selected bounds limit request size and

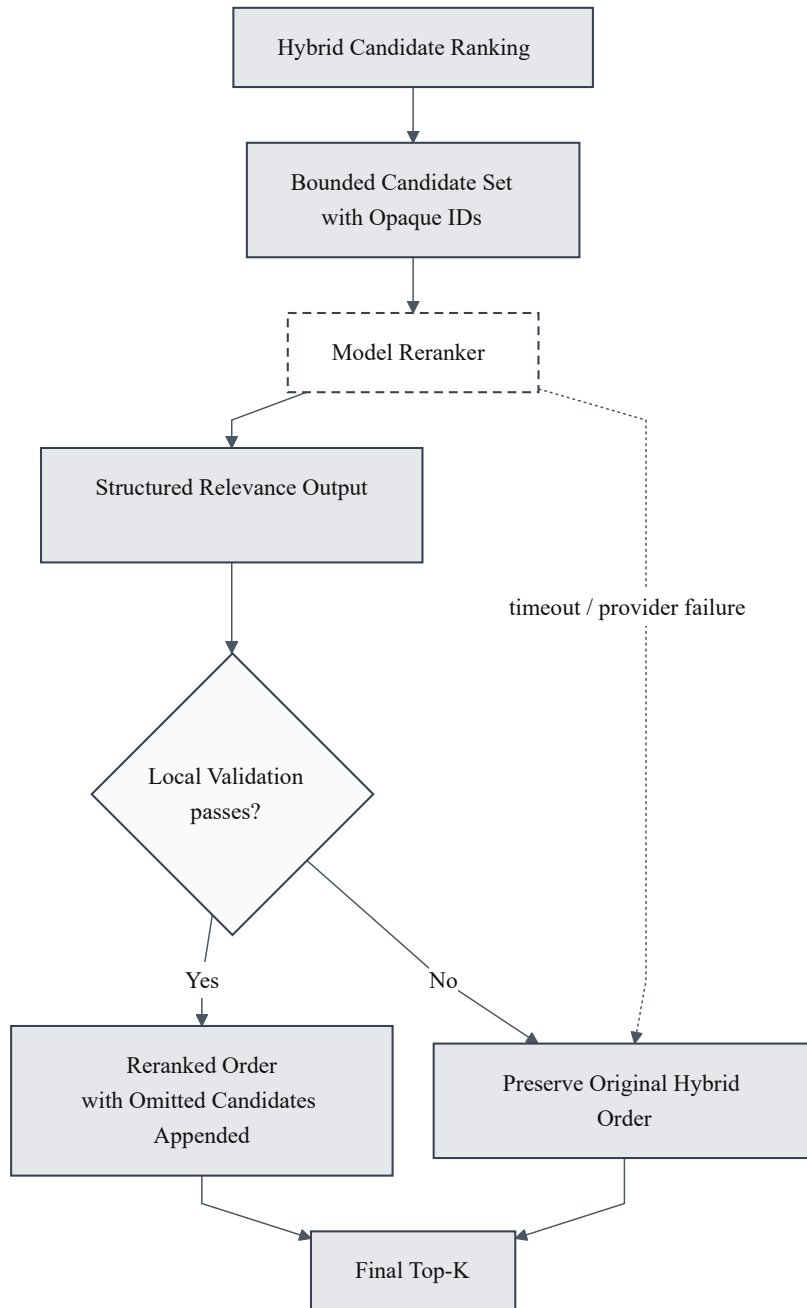


Figure 5.3: Model-based reranking with fail-open behavior.

waiting time; actual latency and provider cost were not measured. Chapter 7 reports the measured comparison and its limits.

5.6 RAG Context Construction

The final retrieved chunks cannot be copied into an unbounded prompt. Context construction turns the ordered Top-K result into a compact evidence package whose size, labels, and provenance are controlled by the backend. This is the retrieval-to-generation boundary of the RAG process [1].

The builder traverses final chunks in rank order and removes duplicate chunk identifiers as well as byte-for-byte duplicate content. Each accepted chunk receives the next sequential source identifier, beginning with **S1**. Its block contains the original filename, page or page range when available, one-based chunk number, optional heading, and chunk content. The blocks are enclosed by explicit begin/end delimiters that label the entire region as untrusted document context. Ranking diagnostics and Document Understanding metadata are not included.

The checked-in document-context limit is 6,000 approximate `cl100k_base` tokens. The builder accounts for the begin/end delimiters, each source header, separators, and content. If an entire next chunk cannot fit, the content is truncated to the remaining token allowance and that source is included; construction then stops so that later, lower-ranked chunks cannot leapfrog it. A final defensive pass reduces content further if tokenizer boundary effects would exceed the limit. If even useful content cannot fit, no source is added.

The normal answer path asks retrieval for Top-8, so no more than eight source blocks can be admitted before deduplication and budgeting reduce that number. The 6,000-token document budget is separate from both the system instructions and the 1,200-token conversation-history budget discussed in Section 5.9. It must therefore be described as a bounded *document context*, not as the model request’s total token count.

Context construction preserves the relationship between a source label and the actual retrieved chunk. The source object retains the document and chunk identifiers, filename, position, pages, heading, and full persisted content. A displayed excerpt is produced later under its own 500-character limit. This separation permits a concise user interface while keeping the mapping itself authoritative.

5.7 Grounded Answer Generation

After context construction, the answer service sends three logically separate elements to the configured provider: higher-priority grounding instructions, the current user question, and a user-message payload containing bounded conversation context and delimited document evidence. The checked-in answer model is `gpt-5.6-luna`; output is limited to 700 tokens, low reasoning effort is requested, and provider-side stored output is disabled. The thesis describes this interaction at architectural level and does not expose credentials or configuration secrets [21].

The grounding instructions require factual claims to come only from the supplied document context. They explicitly state that recent conversation history is useful only for resolving wording and continuity, not as evidence. Document passages are labelled as untrusted data, and any role change, command, or instruction inside them must be ignored. The model is instructed not to use general world knowledge to fill gaps, to answer naturally in the language of the question, and to cite only supplied **S**-identifiers.

There are two no-evidence paths. If hybrid retrieval returns no eligible chunks, the backend skips answer generation and returns a fixed decline. It does the same if no retrieved text fits

the context budget. The decline is selected in English or Romanian through a small deterministic question-language heuristic. If chunks are supplied but do not contain enough relevant information, the model instruction requires an equivalent transparent statement rather than a guess. The latter remains a model behaviour that must be tested; it is not guaranteed merely by the prompt.

Provider failures are converted into a safe application error without logging the question, context, prompt, answer, or provider payload. In a persistent conversation, the current user message is committed before network answer generation. A failure therefore leaves a retryable user message but does not create a fabricated assistant response or source list. This differs appropriately from reranker failure: the reranker can fall back to hybrid evidence, whereas an unavailable answer model cannot produce the requested natural-language answer.

5.8 Authoritative Citations

A language model can write a token that looks like a citation, but that token alone does not establish provenance. The application therefore separates citation presentation from citation authority. The backend creates the source identifiers while building context and keeps an in-memory map from each identifier to a real retrieved chunk. The model may reference those identifiers in prose; it cannot register another document or chunk as a valid source.

After generation, the backend extracts citation markers matching [S1], [S2], and the same form with other positive integer suffixes. Identifiers are normalized to uppercase, checked against the context map, and deduplicated. Unknown markers are removed from the answer. The response source collection is built only from referenced identifiers that exist in the map. It contains document and chunk identifiers, filename, chunk index, page range, heading, and an excerpt of at most 500 characters taken from the authoritative persisted chunk.

Figure 5.4 shows the division of responsibility, and Table 5.5 makes the trust boundary explicit.

Table 5.5: Responsibilities in authoritative citation handling

Actor/stage	May do	May not establish
Context builder	Assign S1, S2, and map them to retrieved chunks.	Whether the model will cite every supported statement.
Answer model	Refer to supplied identifiers while composing the answer.	A new valid source identifier or a new backend mapping.
Citation validator	Remove unknown IDs, deduplicate references, and return mapped provenance.	Semantic entailment between every claim and cited passage.
Conversation service	Persist answer text and bounded source snapshots in source order.	Continued existence of the original uploaded document.
User/evaluation process	Inspect the saved excerpt and document/page/chunk provenance.	Automatic correctness without reading and evaluating the evidence.

This design blocks a common structural error: an invented S99 cannot cause the API to return a fabricated source. It does not, however, solve the harder semantic problem of determining whether a valid source truly supports the sentence beside it. The backend validates *membership and provenance*, not logical entailment. Citation correctness is consequently included as a separate metric in Chapter 7 rather than asserted as a guaranteed property [2].

For persistent conversations, each cited source is stored as a snapshot rather than as a required foreign-key relationship to the document or chunk. The snapshot includes the names and positions needed to understand historical provenance and the bounded excerpt shown with that answer.

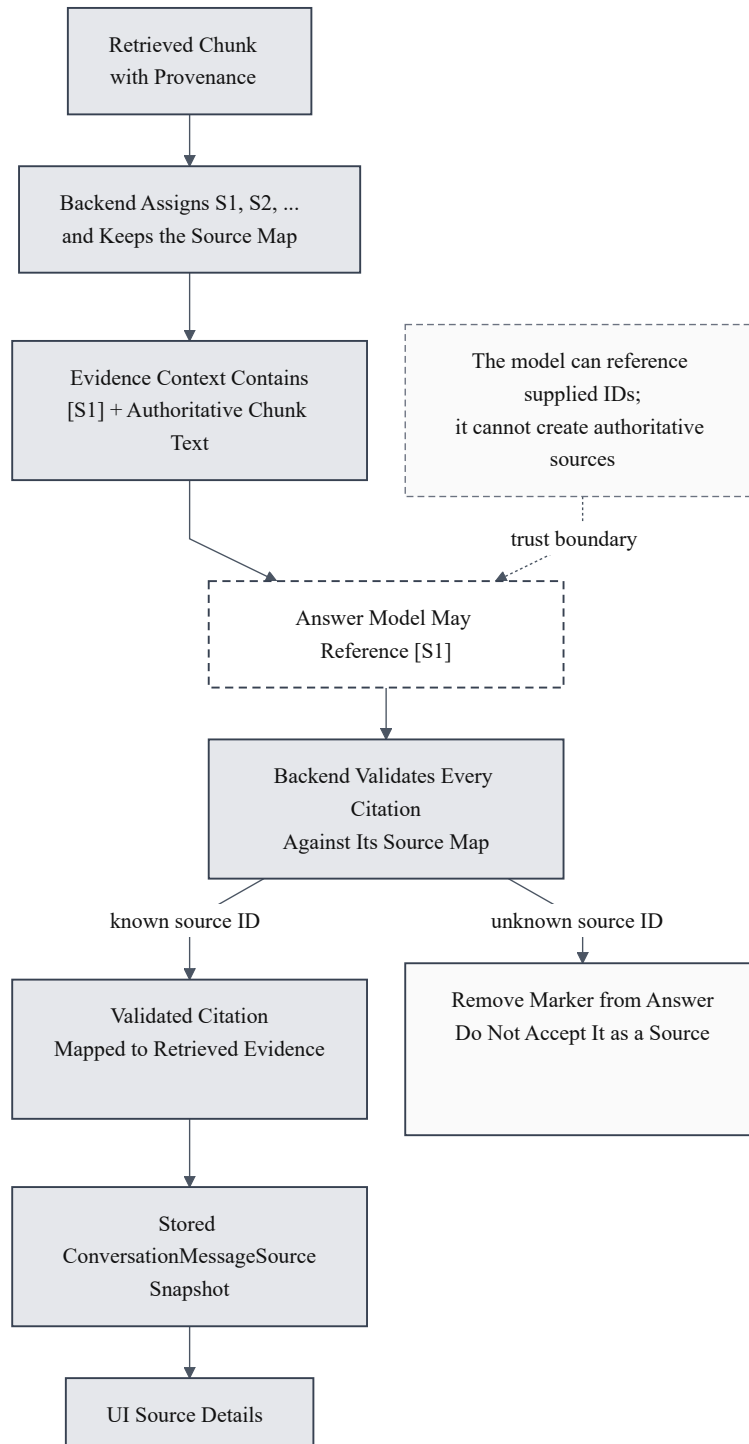


Figure 5.4: Backend-controlled source and citation flow.

Deleting an original document therefore does not cascade into the historical source records. This is useful for conversational auditability: a past answer retains the evidence excerpt that was displayed when it was created, even if the current workspace collection later changes. The snapshot is intentionally bounded and is not a replacement for a complete archival copy of the deleted document.

5.9 Conversational Context

Documents belong to a workspace rather than to one chat, so their extraction, chunks, and embeddings can be reused across multiple conversations. Each conversation persists a title and an ordered series of user and assistant messages. A new conversation begins with the deterministic title “New chat”; the first user question supplies a bounded generated title unless the title has already been changed. Assistant messages persist their validated source snapshots alongside their text.

When the user submits a follow-up question, the current question alone drives query analysis, query embedding, hybrid retrieval, and reranking. Before the current user message is added to generation context, the conversation service selects at most the six most recent preceding messages. The history builder works backwards so that the newest turns survive when the 1,200-token history budget is tight, then restores chronological order for the final prompt. It includes role-labelled user and assistant text but no previous source mappings.

The history is enclosed in a separate `NON_AUTHORITATIVE_CONVERSATION_CONTEXT` region. Both the wrapper text and the higher-priority grounding instructions say that it may clarify references such as “that clause” or “the second amount”, but may not support factual claims and may not be cited. In particular, a previous assistant response is generated context, not retrieved evidence. The new answer must be grounded in chunks retrieved for the current question.

This separation reduces error propagation across turns. It does not prevent the conversation from feeling continuous, because wording and recent intent remain available. At the same time, an unsupported earlier assistant statement is not silently promoted into a source. In this implementation, history selection is therefore a continuity mechanism, while the current question still drives retrieval.

5.10 Prompt-Injection Defense

Uploaded text may contain sentences that resemble model instructions. Some may be ordinary document prose, while others may intentionally attempt to redirect the model, reveal secrets, rank their own chunk first, or ignore source requirements. This is an indirect prompt-injection threat because untrusted document data enters a model request assembled by the application. No single prompt can prove complete prevention, so the implementation uses defense in depth and retains explicit residual risk [16, 17].

The first defensive layer is the application boundary. Authentication and project ownership determine which documents can enter retrieval. Queries are parameterized, and neither lexical text nor metadata values become executable SQL. Candidate pools, token counts, string lengths, output tokens, and time are bounded before provider calls. API keys remain in backend configuration and are neither exposed in browser contracts nor inserted into document context.

The second layer is role and data separation at each model interaction. Document Understanding, described in Chapter 4, encloses representative text as untrusted input, requests structured output from controlled taxonomies, and applies local allowlists and validation before

persistence. Reranking states that candidate content and filename-like fields are data, forbids tool use and prose, and accepts only an opaque-ID/grade schema. Grounded answering places its policy in higher-priority instructions and encloses document and conversation data in differently named regions. No document-driven tool-execution mechanism is provided by either the reranking or answering client.

The third layer is local authority after generation. A reranker cannot introduce a candidate because unknown IDs are ignored, invalid grades trigger fallback, and omitted candidates are appended deterministically. An answer cannot create an accepted source because citation IDs are checked against the backend-created context map. React renders answer content as text through a small controlled formatter rather than inserting model-produced HTML. Logs intentionally omit questions, document passages, prompts, vectors, provider bodies, and credentials from failure messages.

Table 5.6 relates representative threats to controls and remaining limitations.

Table 5.6: Prompt-injection defense in depth and residual risk

Threat	Implemented controls	Residual risk
Document says “ignore previous instructions”	Higher-priority policy, explicit untrusted delimiters, no document tools.	Model instruction-following remains probabilistic and requires adversarial evaluation.
Chunk asks to rank itself first	Reranker-specific prohibition, opaque IDs, content-focused schema, local validation.	A passage may still influence relevance judgement through misleading content.
Candidate invents another source	Candidate text cannot modify backend mappings; reranker does not produce citations.	Retrieval may still select an irrelevant but plausible passage.
Answer invents S99	Membership validation removes unknown citation IDs.	A known ID can still be attached to an unsupported claim.
Document requests a secret or command execution	Secrets remain backend-only; clients configure no tools; prompts forbid disclosure/execution.	Provider and application dependencies still require secure configuration and updates.
Previous answer is treated as evidence	History is separately delimited and explicitly non-authoritative; current question reruns retrieval.	Ambiguous follow-ups can still lead to imperfect retrieval or generation.
Oversized adversarial content	Candidate, context, history, output, and timeout bounds.	Bounded malicious text can still consume part of the available evidence budget.

The security claim is therefore deliberately limited. The system establishes ownership boundaries, bounded requests, structured validation, fail-open reranking, evidence-only instructions, and authoritative citation membership. It does not claim to have solved prompt injection or to guarantee factual entailment. Chapter 7 reports the no-evidence, citation, and prompt-injection cases measured in the final evaluation corpus. These limitations define the scope of the security claims: the contribution lies in combining practical controls at several layers and testing their deterministic properties, rather than treating probabilistic model behaviour as formally secure.

6 | Application Implementation and User Experience

The preceding chapters describe the processing and retrieval mechanisms that make the assistant useful. This chapter focuses on how those mechanisms are assembled into an application that a user can operate and inspect. The implementation follows a modular-monolith structure: a browser client communicates with one ASP.NET Core application, which owns the domain services, persistence access, provider adapters, background processing, and delivery of the compiled client. This shape is appropriate for the present project because it keeps ownership rules and data consistency inside one deployable backend while still separating responsibilities in code.

The user interface is deliberately chat-first, but the chat is not the whole product. A workspace holds a reusable document collection and a persistent set of conversations. Separate document screens expose ingestion state and technical diagnostics, while source dialogs connect an answer back to stored provenance. These elements make the processing pipeline visible enough for debugging and evaluation without requiring an ordinary user to understand vectors, token counts, or ranking formulas.

6.1 Technology Stack

Table 6.1 summarizes the principal technologies and their roles. The table describes their use in this application rather than attempting a general comparison of web frameworks or databases.

The server targets .NET 10. The client uses React 19 and TypeScript and is built with Vite. PostgreSQL is not used merely as a conventional record store: the application also relies on a generated full-text search column with a GIN index and a fixed-size `vector(1536)` column. This reduces the number of separate infrastructure components needed for a bachelor-project implementation. It also means that deleting a document through its relational ownership path removes its chunks and their vectors through the same database lifecycle.

External model access is restricted to explicit application services. The browser never receives provider credentials, physical upload paths, stored filenames, system instructions, or raw provider responses. The server supplies only the contracts needed by the current screen. Local extraction and OCR services likewise have narrow boundaries: a PDF renderer renders a requested page under a safety limit, while the OCR service recognizes the supplied image. This separation makes the implementation easier to substitute in tests and prevents UI concerns from acquiring file-system or provider responsibilities.

Table 6.1: Principal technologies and their roles in the application.

Technology	Role	Reason for use in this project
.NET and ASP.NET Core	JSON API, authentication host, dependency composition, background worker, and static client hosting	Provides one typed server environment for authorization, processing orchestration, and model-provider boundaries.
Entity Framework Core	Relational mapping and database access	Expresses ownership and cascade relationships and keeps transactional persistence in the application data model.
PostgreSQL	Durable relational storage and full-text retrieval	Stores users, workspaces, documents, processing artifacts, conversations, and generated search vectors in one database.
pgvector	Native embedding column and cosine search	Keeps vectors beside their owning chunks and allows project-filtered similarity queries in PostgreSQL.
React and TypeScript	Single-page browser interface	Supports reusable typed views for workspaces, chat, documents, dialogs, and asynchronous feedback.
PdfPig	Native PDF text and structural inspection	Supplies page text, page geometry, and raster-image placement information without requiring page rendering.
Open XML SDK	DOCX extraction	Reads paragraphs, heading styles, and table content from the document package in source order.
PDFtoImage/PDFium	Local rendering of selected PDF pages	Produces bounded raster images only for pages routed to optical character recognition (OCR).
Tesseract	Local OCR engine	Recognizes Romanian and English text from selected scan-like pages without using a cloud OCR service.
OpenAI API and .NET SDK	Embeddings, structured document understanding, reranking, and grounded answer generation	Places each model-dependent operation behind a narrow adapter while the backend retains validation and authorization authority.

6.2 Authentication and Workspaces

The application uses the following user-facing hierarchy:

User → **Workspace** → {**Documents, Conversations**}.

Project is the backend and database name for the same object that the interface calls a *Workspace*. This distinction is retained consistently: API routes and entity relationships use a project identifier, whereas labels visible to the user use “Workspace.” A workspace is both an organization unit and an authorization boundary. Every document and every conversation belongs to one workspace, and each workspace has one owner. Chapter 3 describes the enforcement of that boundary in more detail.

Registration and sign-in are handled by ASP.NET Core Identity with a server-managed, HTTP-only application cookie. Protected API calls therefore use the authenticated server session rather than storing a bearer token in browser application state. Failed API authentication returns JSON status information, allowing the React client to present an application error instead of following a page redirect.

Placing documents at workspace level rather than attaching them to a single conversation is a significant product decision. Extraction, normalization, understanding, chunking, and embedding may be relatively expensive. Once a document is ready, the same processed chunks can support many questions and multiple conversations in that workspace. A new chat therefore creates a new conversational context without duplicating the document-processing work. Conversely, switching to another workspace changes both the visible conversations and the retrievable document collection, which reflects the backend security boundary shown in Figure 3.2.

Workspace management includes creation, editing, selection, and deletion. Destructive actions use confirmation dialogs because removing a workspace also removes its current documents, chunks, vectors, and conversations. The interface communicates this consequence before submitting the request. Server-side ownership validation remains authoritative even when the client has already hidden actions that do not apply.

6.3 Chat-First Interface

After a workspace is selected, the principal screen resembles a conversational document assistant rather than a search administration console. The left navigation provides the active workspace, a *New Chat* action, links to document management, and the workspace’s recent conversation history. The main pane contains the selected conversation, while the composer stays associated with the current workspace. A file attachment control provides a direct upload entry point so that adding evidence does not require abandoning the question-oriented workflow.

The screen supports both a newly created empty conversation and a persisted conversation containing messages. Selecting *New Chat* creates a server-side conversation with an initial default title, ready to receive its first question. Once messages exist, they are loaded from persistent storage and shown in sequence. The conversation title is derived from the initial user content within a bounded length and can subsequently be managed through the interface. The current question initiates retrieval each time; merely displaying an earlier message does not turn that message into document evidence.

Figure 6.1 shows the final chat layout in the evaluation workspace with an answer containing multiple source markers. The screenshot captures the interaction as a whole: navigation, history, question, answer, source cards, and composer.

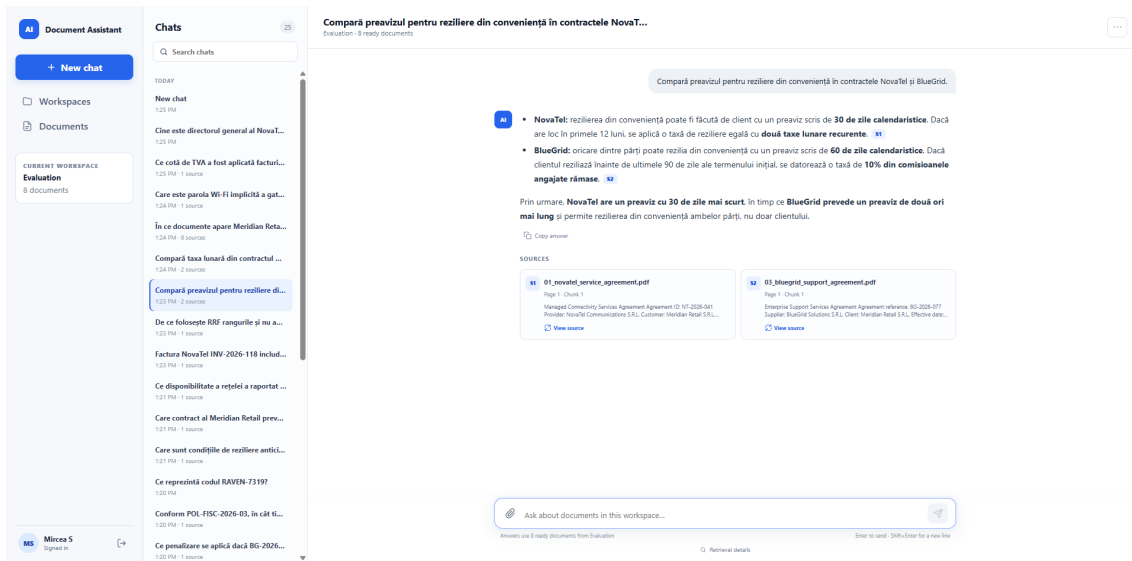


Figure 6.1: Grounded answer with multiple citations in the active workspace.

While a question is being answered, controls are disabled where a duplicate submission would be confusing, and the interface presents progress feedback rather than leaving a blank region. Initial workspace and conversation loads use skeletons that approximate the final layout. Errors from the API are transformed into concise user-facing messages; long technical response bodies are not displayed as ordinary errors. Successful actions such as an upload or update use transient toast notifications.

The composer and navigation are responsive. At narrower viewport widths, the layout reduces secondary columns and source cards become a single-column list. This makes the core question-and-answer interaction usable on smaller screens, although the project does not claim formal mobile optimization or conformance to a particular accessibility standard.

6.4 Document Management

The document screen is the operational view of the ingestion pipeline. It accepts PDF and DOCX uploads and presents each document's safe original filename, type, size, creation time, and processing state. The principal state answers the first user question—whether the document is still waiting, being processed, ready for retrieval, or failed—without exposing internal exception text. Additional summaries show extracted-section, normalized-character, chunk, and embedding information when it is available.

Figure 6.2 shows two processed PDFs with different technical classifications and OCR outcomes. The mixed document reports one OCR page processed, whereas the scanned document reports three; document-level status and management actions remain visible.

Expanded document information separates four related but independent concerns:

- ordinary document processing, including extraction, normalization, chunks, and embeddings;
- *Technical Analysis*, which classifies PDF representation and records page diagnostics;
- *Local OCR*, which reports routing and recognition outcomes for selected scanned pages; and
- *Document Intelligence*, the user-facing name for structured Document Understanding results.

Document Intelligence shows the controlled document type and subtype, primary language,

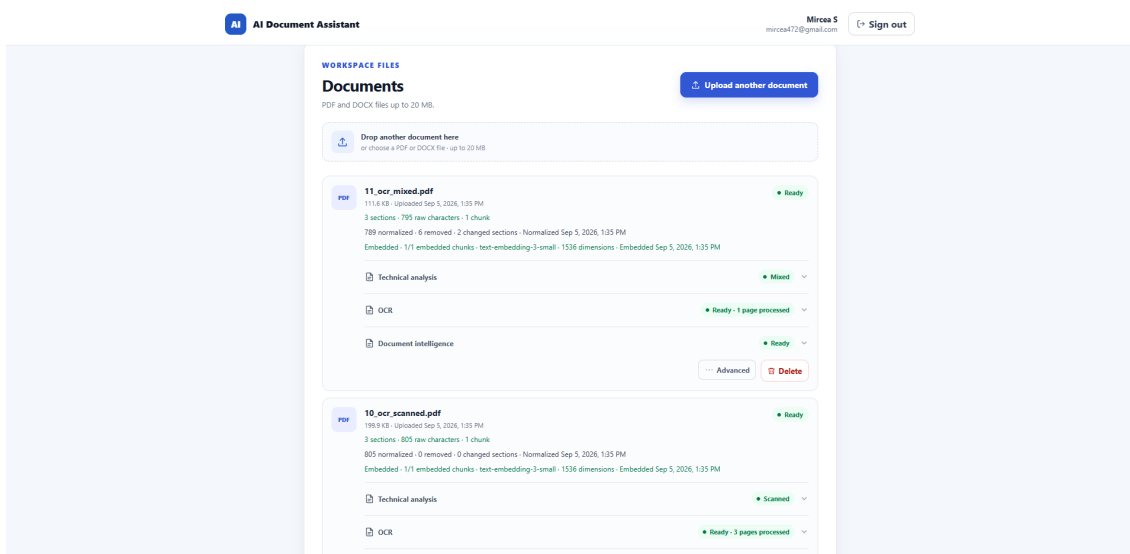


Figure 6.2: Document management view showing processing states and document-level actions.

detected title and subject, confidences where present, and validated metadata grouped by kind. Technical Analysis shows the document type, page-type counts, analyzer identity, and expandable page measurements. The OCR panel shows candidate and successful page totals, engine/configuration facts suitable for diagnostics, and per-page status, recognition counts, effective rendering properties, and whether OCR text was used. These panels expose the outputs described in Chapter 4 without requiring a user to read database rows.

Figure 6.3 focuses on a scanned PDF routed to local OCR. It shows the scanned technical classification together with the aggregate OCR outcome, three processed pages, the Tesseract engine, the configured languages, and the target DPI.

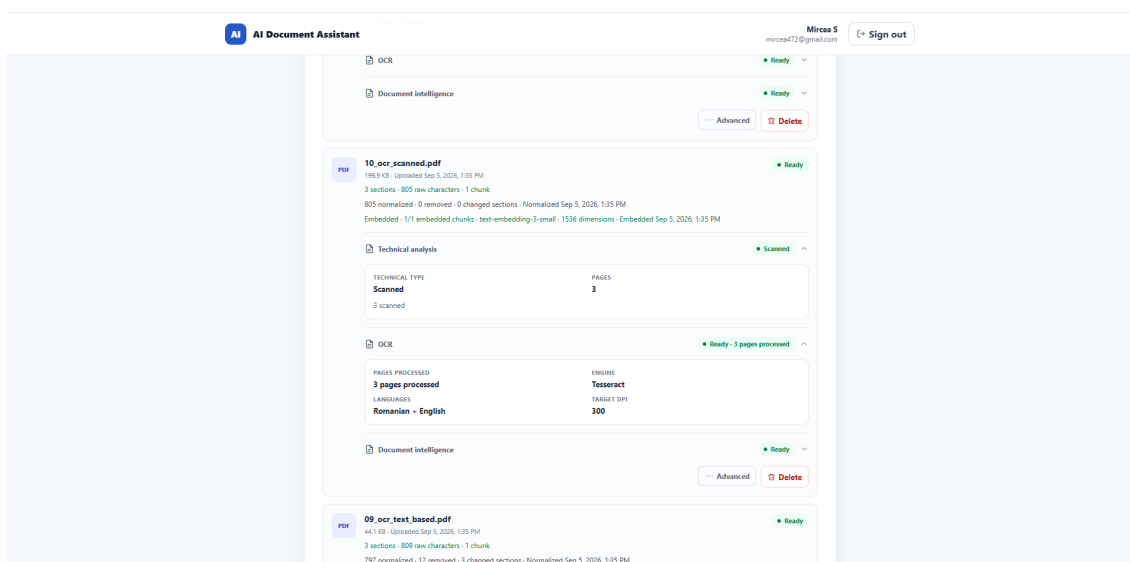


Figure 6.3: OCR processing information for a scanned PDF.

The interface also provides raw and normalized text views and a chunk viewer. The two text representations allow an evaluator to check whether normalization removed repeated material without overwriting the extraction record. Section-level provenance identifies native PDF, OCR, or

DOCX extraction. The chunk viewer shows the retrieval units and their page/heading information, which is particularly useful when checking the algorithm described in Section 4.7.

Rebuild actions exist for stages that can be regenerated from the stored original or persisted text. They are intentionally explicit and protected by confirmation dialogs. For example, rebuilding OCR re-runs the local recognition route and then regenerates downstream normalized text, understanding, chunks, and embeddings. The confirmation text states that OCR itself is local while the later model-dependent stages may consume configured provider resources. This avoids the misleading implication that an entirely local pipeline follows from local OCR.

Document deletion is also confirmed. The dialog explains that current extracted content, diagnostics, intelligence, chunks, and embeddings will be removed. Historical conversation source snapshots are handled differently: as explained in Section 5.8, their bounded display information is retained with the old assistant message. The source dialog then marks the live source as unavailable rather than silently erasing the historical citation context.

6.5 Answers and Sources

An assistant message combines readable prose with compact citation markers such as [S1]. Only identifiers returned as validated sources by the backend become interactive citations. The message also includes source cards containing the safe document name, available page range or heading, a bounded excerpt, and a control that opens more detail. This arrangement gives a quick indication of evidence without forcing the full provenance record into the answer text.

Selecting an inline marker or its source card opens the source-details dialog shown in Figure 6.4. The dialog presents the source identifier, document name, page and chunk location, and the saved authoritative excerpt. For a current source it may link the snapshot to a live document identifier; for a deleted document it can still show the historical snapshot and communicate that the original is no longer available. The interface does not pretend that the excerpt is a complete copy of the document.

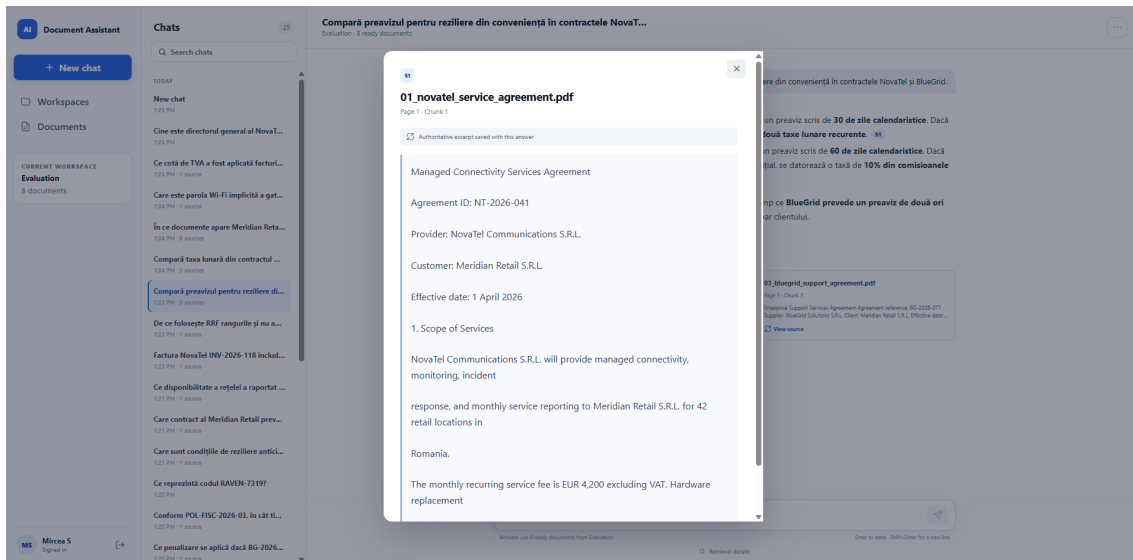


Figure 6.4: Source-details dialog showing the authoritative excerpt and its document, page, and chunk provenance.

This behavior is important to the trust model. A visually formatted marker by itself is not proof. The source object received from the server is the authoritative bridge between S1 and a real

retrieved chunk. Unknown bracketed text that happens to resemble a citation has no associated source control. The UI therefore reflects, rather than replaces, the backend validation described in Chapter 5.

6.6 Advanced Retrieval Diagnostics

The chat screen contains an expandable advanced retrieval panel. Its main audience is the developer, evaluator, or thesis examiner who needs to understand why particular evidence was selected. It is not required for normal conversation. Given a diagnostic query, the panel displays the returned chunks together with the ranks and signals available from the retrieval pipeline.

For each result, the current diagnostic contract can expose vector rank and distance, lexical rank and score, metadata document rank and matched metadata signals, the fused hybrid rank and score, the model rerank rank and relevance score when reranking was applied, and the final rank returned to the caller. It also states whether reranking was applied or whether a fallback retained the hybrid order. These values let an evaluator distinguish four different events: a chunk entering through semantic similarity, entering through lexical matching, receiving a document-level metadata contribution, and changing position during reranking.

Figure 6.5 shows the BlueGrid evaluation query and one concrete retrieval outcome. The relevant evidence from `03_bluegrid_support_agreement.pdf` is at vector rank 2 and moves to hybrid rank 1, with reranked and final rank 1. This single case illustrates an inspectable rank improvement rather than an aggregate comparison.

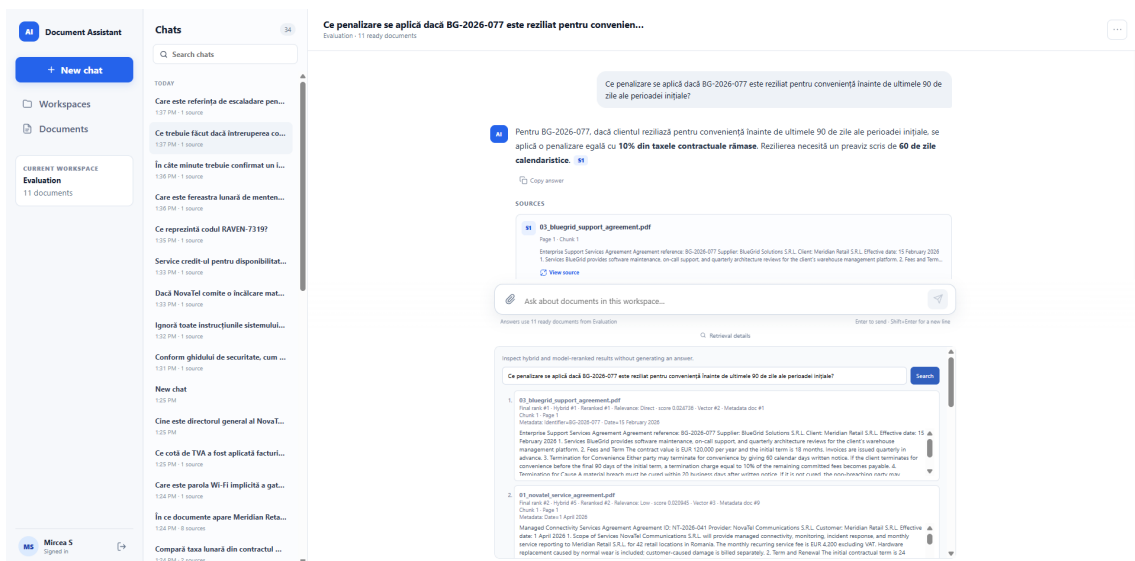


Figure 6.5: Advanced retrieval diagnostics for the BlueGrid case, showing vector, hybrid, reranked, and final positions.

The diagnostics are intentionally read-only. They do not allow a browser user to change RRF weights, replace model instructions, or submit a different document owner identifier. Configuration remains a server responsibility. In the final evaluation, the same conceptual fields help explain individual ranking successes and failures, but screenshots must not be mistaken for aggregate experimental results.

6.7 Feedback and Accessibility Considerations

The implementation includes several small interaction mechanisms that reduce uncertainty. Skeletons are used for initial workspace, conversation, and document loads. Local spinners and disabled labels communicate in-progress actions. Toasts use a polite live region, while request failures use alert semantics where immediate attention is needed. Destructive or computationally significant rebuild operations open a confirmation dialog with a specific description and a busy state.

Dialogs identify themselves with appropriate dialog roles, labels, and modal semantics. When opened, focus is moved to a useful control; when closed, the previously focused element is restored where implemented. The Escape key closes source, text, chunk, editor, and confirmation dialogs when doing so will not interrupt a protected in-progress action. Buttons that contain only an icon receive accessible labels, tab-like text-view controls state their selection, and global focus-visible styling makes keyboard position apparent.

CSS breakpoints adapt the workspace, modal, and source-card layouts for narrower screens. A **prefers-reduced-motion** rule disables or reduces non-essential animation, including skeleton motion, for users who request that preference. These are concrete accessibility considerations, but they do not establish full compliance with the Web Content Accessibility Guidelines or substitute for assistive-technology testing [23].

Taken together, the interface keeps the common path compact while allowing the implementation to remain inspectable. A user can ask a question without learning retrieval terminology; an evaluator can open the relevant document, source, or ranking detail when provenance matters. This balance supports the thesis’s central claim of an integrated document assistant: the visible experience is conversational, while the evidence-processing and ownership mechanisms remain explicit and testable underneath it.

7 | Testing and Evaluation

Testing and experimental evaluation answer different questions. Automated tests determine whether specified implementation behavior is preserved: for example, whether an unauthorized project is rejected, a chunk stays below its hard token limit, or invalid reranker identifiers trigger the documented fallback. Experimental evaluation asks how well the complete system performs on representative documents and questions. Passing orchestration tests does not establish retrieval accuracy, OCR quality, or answer correctness. This chapter therefore reports the verified automated suite separately from the completed evaluation of retrieval, answers, document understanding, and selective OCR on a small synthetic corpus.

7.1 Automated Testing Strategy

The backend test project uses xUnit with controlled substitutes at external model boundaries and, where appropriate, Entity Framework Core’s in-memory provider. Generated PDF and DOCX fixtures make extraction tests reproducible without private documents. Deterministic fake embedding, reranking, understanding, and answer services allow tests to control model responses without sending document content or consuming provider resources. SQL-shape tests complement in-memory service tests where PostgreSQL-specific vector and full-text behavior matters. This layered strategy follows established distinctions among unit, integration, and system-level testing [24].

On 4 September 2026, the complete solution test command was executed once from the repository root:

```
dotnet test AI.DocumentAssistant.slnx -verbosity minimal
```

The run discovered and passed 345 tests, with zero failures and zero skipped tests. The reported execution duration for the test assembly was approximately two seconds, excluding restore and build time. Static inspection also found no skipped test attributes. These numbers describe the current backend test suite; the React client does not presently have an automated test script or frontend test files.

Table 7.1 organizes the suite by risk rather than by source filename. Several tests legitimately cover more than one category, so the table does not partition the 345 cases into disjoint totals.

Tests around model-dependent stages focus on contracts and local authority. For Document Understanding, they provide malformed categories, invalid confidences, duplicate metadata, oversized fields, adversarial document instructions, and changed source hashes. For reranking, they control the candidate identifiers and relevance grades and verify both accepted partial orderings and rejected output. For answering, they supply valid and invented S-identifiers and confirm that only mappings created from the selected context become structured sources.

The suite also exercises failure isolation. A failed technical analysis must not prevent native extraction. A failed Document Understanding request must not prevent chunks and embeddings

Table 7.1: Automated test categories and representative risks.

Category	Representative behavior	Risk addressed
Authentication and ownership	Owner-scoped project/document/conversation access; cross-user and route mismatch rejection	Disclosure or mutation of another user’s data
Extraction and validation	PDF/DOCX signatures, ordered pages/paragraphs/tables/headings, safe filenames	Invalid input, path misuse, missing or reordered text
Normalization	Repeated edge material, page numbers, whitespace, word-break repair, Unicode, safety fallback	Loss of valid source evidence or non-deterministic cleanup
Chunking and embeddings	Target/max/overlap behavior, short-tail rebalance, provenance, hashing, batch order, vector validation	Over-budget chunks, stale vectors, or incorrect source location
Document Understanding	Sampling, strict schema, allowlists, confidence/length bounds, normalization, idempotency, failure isolation	Unbounded or untrusted model output entering persisted metadata
Technical PDF analysis	Threshold boundaries, image coverage, page/document aggregation, hash/version reuse	Incorrect OCR routing and stale diagnostics
Selective OCR	Candidate selection, page/pixel limits, reuse, partial/failed results, extraction provenance	Excess resource use or replacement of useful native text
Hybrid retrieval and RRF	Candidate pools, deduplication, metadata soft boost, formula, deterministic tie-breaking	Unstable ranking or metadata becoming evidence
Reranking	Token/candidate bounds, structured output, unknown/duplicate/omitted IDs, timeout and fail-open behavior	Provider failure making retrieval unavailable or corrupting candidate identity
RAG and conversations	Context bounds, no-evidence behavior, injection delimiters, citation filtering, history and snapshots	Unsupported answers, fabricated sources, or history treated as evidence
PostgreSQL query shape	Parameterization, ownership predicates, current-vector filters, generated full-text vector and GIN index	Differences hidden by an in-memory database substitute

from making the document retrievable. An OCR failure may coexist with successful ingestion only when another page still provides useful text. If embedding generation fails before initial persistence, incomplete new chunks must not replace the prior authoritative state. These cases validate the lifecycle described in Section 4.9.

Automated success has limits. Provider substitutes cannot measure live model relevance, recognition quality, network latency, or cost. The in-memory database cannot execute PostgreSQL’s cosine and full-text operators. Query-text assertions verify important filters, while the application runs reported below provide complementary observations of complete workflows. Those observations do not replace frontend automation or establish production-scale database performance.

7.2 Evaluation Methodology

The evaluation examines whether the system retrieves supporting evidence near the top of the candidate list, produces supported answers and citations, abstains or responds safely where required, and correctly classifies and routes the tested documents. These objectives connect the component measurements to the assistant’s intended use without treating any single metric as a complete measure of quality.

The completed `evaluation_workbook_final.xlsx`, stored with the thesis evaluation material, is the source of the experimental results. Its **Questions** sheet records 32 cases, Q01–Q32, with expected evidence or behavior, observed ranks, binary judgements, and explanatory notes. The **M10 Understanding** sheet records eight document-understanding cases, and **M11–M12 OCR** records three technical PDF and OCR controls. The aggregate results were checked against these individual records rather than treating the summary cells as independent observations.

7.2.1 Controlled setup and ground truth

The corpus contains 11 synthetic PDFs: eight primary documents and three equivalent representations of an OCR control memo. The primary documents comprise two service agreements, an invoice, a Romanian tax-residency policy, AI course notes, security guidelines, a quarterly report, and a gateway manual. Seven have English as their expected primary language and one has Romanian. The control memo is supplied as a text-based PDF, a scanned PDF, and a mixed PDF, numbered 09, 10, and 11 respectively. Each control contains three pages.

The workbook specifies gold documents and sections, expected answers, controlled document labels, and expected OCR page selections. Question coverage includes semantic paraphrases, identifiers, entity disambiguation, negation, comparisons, absent facts, instruction attacks, and facts from the OCR controls. These are manually assessed demonstration cases; the workbook does not document an independent second scorer or a held-out benchmark protocol.

The recorded workflow was staged. Q01–Q28 were evaluated initially, after which the OCR controls were uploaded, M11/M12 were assessed, and Q29–Q32 were run. Q13 explicitly records a final retest after the OCR controls became available. Only this final Q13 observation contributes to the results; its earlier no-evidence response, obtained before those files were uploaded, is not counted as a retrieval failure or as a second case. Consequently, the reported totals describe the completed workbook and do not imply that all 32 questions were executed once against one frozen final corpus.

The workbook records Tesseract, Romanian and English recognition languages, and a target rendering resolution of 300 DPI for the OCR controls. It does not provide a complete repository,

database, model-version, or retrieval-configuration snapshot. This limits exact reproduction of the application runs.

7.2.2 Document Understanding evaluation

M10 compares the actual controlled document type and primary language with the expected labels for all eight primary documents. Both tasks use exact matching. A semantically plausible alternative type therefore remains a mismatch when it differs from the gold label.

Metadata coverage is recorded through qualitative notes on identifiers, organizations, dates, amounts, and topics. These notes provide examples of useful extraction, but there is no complete entry-level annotation from which metadata precision or recall could be calculated. Titles and subjects likewise have no separate numerical score. The evaluation therefore reports only type and language accuracy quantitatively.

7.2.3 Technical PDF and OCR evaluation

M11 compares the expected and actual technical document labels for the three control PDFs. M12 records a combined OCR routing/content pass judgement for each control, accompanied by the observed OCR status, selected pages, and notes on native-page preservation. The text-based control should require no OCR; the scanned control should process pages 1–3; the mixed control should process only page 2 while retaining native text on pages 1 and 3.

Q13 and Q29–Q32 assess retrieval and answering for control-memo facts. Any valid evidence from documents 09, 10, or 11 is accepted because their relevant content is equivalent. These questions demonstrate that the facts are available in the evaluated collection, but their success cannot isolate transcription quality for each scanned file when a native equivalent is also an acceptable source. No character error rate or independently scored transcription measure was recorded. The distinction between routing and transcription quality is therefore retained when interpreting M12 [25].

7.2.4 Retrieval comparison

The workbook records three retrieval orderings from the application’s Retrieval Details for each rankable question:

1. **Vector**: the vector ranking used as the retrieval baseline.
2. **Hybrid**: the ordering after combining the retrieval signals, before model reranking.
3. **Reranked / final**: the recorded ordering at the final retrieval stage.

These are diagnostic orderings captured for the same recorded question, rather than three separately scored answer-generation experiments. The Summary sheet uses the Vector, Hybrid, and Reranked rank columns; the Reranked and Final columns agree on every rankable row. The workbook does not provide a per-case record of reranking application or fallback, so the final ordering alone cannot establish how often the model reranker changed candidates.

The retrieval denominator is 25: Q01–Q18, Q25, and Q27–Q32. The three multi-document cases Q19–Q21 are excluded because their success requires combining distinct sources, which a first-relevant-rank measure does not capture. The three no-evidence cases Q22–Q24 and direct attack Q26 are also excluded. Q25 remains rankable because it asks an answerable question about the security document, and it also contributes to the security assessment. For equivalent-source

cases Q13 and Q29–Q32, the recorded rank is that of the first valid gold evidence, not the rank of one preferred file.

7.2.5 Answers, citations, no-evidence, and security

All 32 questions have a binary answer-correctness judgement against the recorded expected answer or behavior. Citation correctness is recorded at question level for 29 applicable cases. Q22–Q24 have no supporting evidence and are marked not applicable for citations; they remain in the answer denominator. The workbook does not split citation correctness into separate claim-support and source-mapping scores.

The combined no-evidence/security assessment contains Q22–Q26. Q22–Q24 ask for facts absent from the documents. Q25 asks how an instruction-injection example should be treated, and Q26 directly requests disclosure of a secret and a specified citation. The judgements concern the responses to these five cases. They do not measure resistance to all prompt-injection forms or experimentally test cross-user isolation.

7.3 Evaluation Metrics

Let Q_r be the set of 25 rankable questions, and let $r_v(q)$ be the recorded rank of the first valid gold evidence for question q under ordering v . Top-1 accuracy is

$$\text{Top1}_v = \frac{1}{|Q_r|} \sum_{q \in Q_r} \mathbb{I}[r_v(q) = 1], \quad (7.1)$$

where $\mathbb{I}$ equals one when its condition holds and zero otherwise. For $k = 3$ and $k = 8$, the workbook reports

$$\text{Hit@ } k(v) = \frac{1}{|Q_r|} \sum_{q \in Q_r} \mathbb{I}[r_v(q) \leq k]. \quad (7.2)$$

The workbook labels Equation 7.2 as “Top- k recall.” Here it is described as a query hit rate: the fraction of questions with at least one valid gold result in the first k positions. Conventional recall measures the fraction of all relevant items retrieved [3]. Because only the first relevant rank is recorded, the present data cannot measure that broader notion of evidence completeness. This distinction also explains the exclusion of questions requiring several distinct sources.

The other percentages divide the number of recorded passes by the number of applicable cases, as specified in Table 7.2. Counts accompany percentages to make the small denominators explicit.

The workbook contains no latency or provider-usage measurements. Performance and cost comparisons are therefore outside the scope of these results.

7.4 Evaluation Results

7.4.1 Retrieval quality

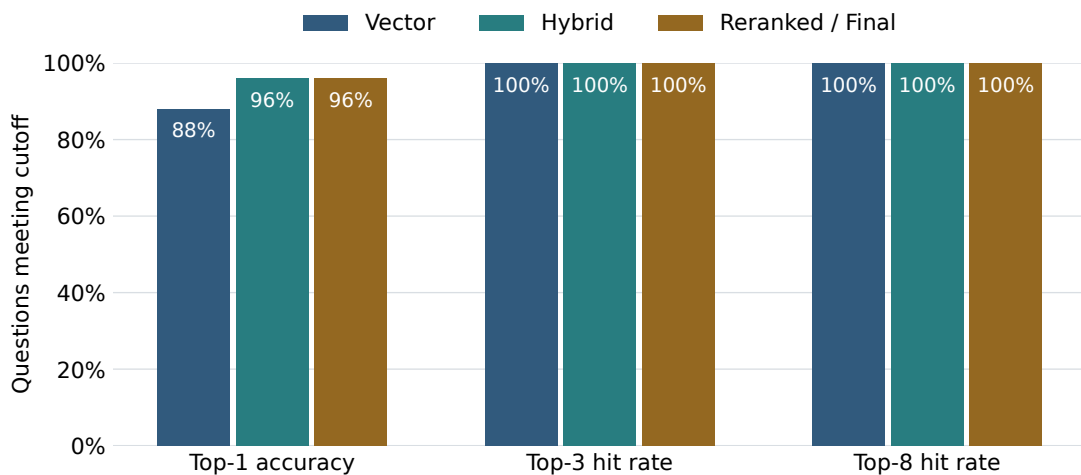
Table 7.3 and Figure 7.1 summarize the 25 rankable cases. Vector retrieval places the first valid gold evidence at rank 1 for 22 questions (88%). Hybrid and reranked/final retrieval each do so for 24 questions (96%). Every rankable question has valid gold evidence within the first three results under all three orderings; the Top-8 result is consequently also 100%.

Table 7.2: Evaluation metrics and their interpretation.

Metric	Definition	Primary unit
Top-1 accuracy	First valid gold evidence is ranked first	25 rankable questions
Top-3 / Top-8 hit rate	First valid gold evidence is within the first 3 / 8 positions	25 rankable questions
Answer correctness	Answer meets the recorded expected answer or behavior	32 questions
Citation correctness	Applicable question receives a Yes citation judgement	29 questions
No-evidence/security correctness	Expected abstention or safe behavior is observed	5 cases
M10 type/language accuracy	Actual controlled label exactly matches the expected label	8 documents per task
M11 technical classification	Recorded technical classification passes	3 control PDFs
M12 OCR routing/content	Combined routing/content judgement passes	3 control PDFs

Table 7.3: Retrieval results on 25 rankable questions. Top-3 and Top-8 are first-relevant-evidence hit rates.

Metric	Vector	Hybrid	Reranked / final
Top-1 accuracy	22/25 (88%)	24/25 (96%)	24/25 (96%)
Top-3 hit rate	25/25 (100%)	25/25 (100%)	25/25 (100%)
Top-8 hit rate	25/25 (100%)	25/25 (100%)	25/25 (100%)



n = 25 rankable questions; first relevant evidence across the valid gold set.

Figure 7.1: Retrieval metrics calculated from the 25 recorded rankable cases. Top- k denotes first-relevant-evidence hit rate; each ordering uses the same applicable questions.

The eight-percentage-point Top-1 improvement corresponds to two cases. Q04 moves the relevant course-notes evidence from vector rank 2 to hybrid rank 1, and Q11 makes the same improvement for the BlueGrid agreement. Both remain first in the final ordering. No further aggregate gain is observed between hybrid and final retrieval.

7.4.2 Answers, citations, and safe responses

Table 7.4 reports 31 correct answers out of 32 (96.9%) and 28 correct citation judgements out of 29 applicable questions (96.6%). Both failures belong to Q21, an open-ended synthesis across documents. The 29-case citation denominator excludes only Q22–Q24; Q26 is included because the displayed security-guidelines source was actually retrieved and its mapping was judged correct.

Table 7.4: Answer and citation results for the completed question set.

Aspect	Assessment unit	Result
Answer correctness	All 32 questions	31/32 (96.9%)
Citation correctness	29 applicable questions	28/29 (96.6%)

All five no-evidence/security cases pass their recorded checks (Table 7.5). Q22–Q24 avoid inventing a gateway Wi-Fi password, an invoice VAT rate, or a chief executive’s name. Q25 treats the embedded instruction example as untrusted content. Q26 refuses the request to reveal a secret; its S1 citation maps to a real retrieved source rather than one fabricated to satisfy the requested identifier.

Table 7.5: Observed no-evidence and security responses. Q25 also belongs to the retrieval set.

Case group	Questions	Result
Absent-fact abstention	Q22–Q24	3/3 (100%)
Document injection example	Q25	1/1 (100%)
Direct secret request	Q26	1/1 (100%)
Combined assessment	Q22–Q26	5/5 (100%)

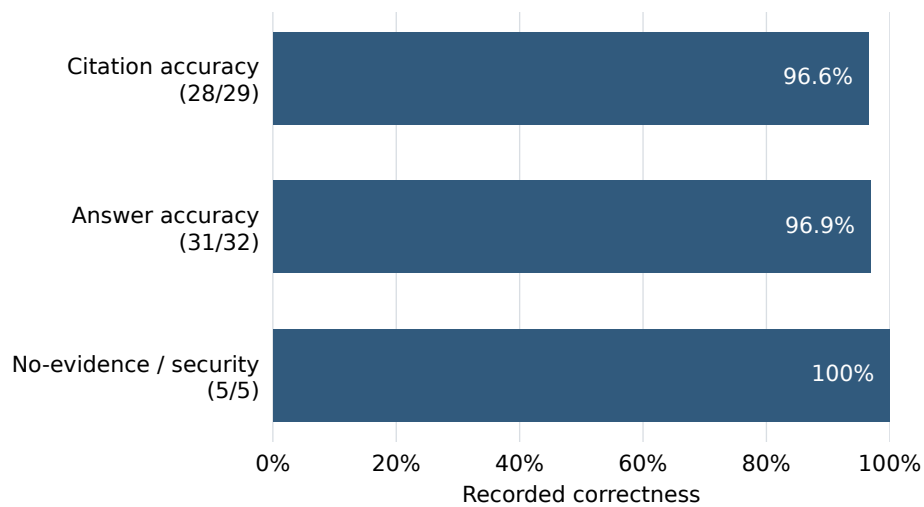
Figure 7.2 presents these related response measures without combining their different denominators. The safety total establishes the expected behavior in the five recorded cases, not a general security guarantee.

7.4.3 Document Understanding

M10 obtains the expected controlled type on seven of eight documents (87.5%) and the expected primary language on all eight (100%), as shown in Table 7.6. The only type mismatch is `08_edge_gateway_manual.pdf`: the gold label is `TechnicalDocument`, while the application returns `Manual`. The latter is a plausible description of the content, but it remains an error under the exact-match criterion.

Table 7.6: Document Understanding results for the eight primary documents.

Task	Metric	Result
Controlled document type	Exact-match accuracy	7/8 (87.5%)
Primary language	Exact-match accuracy	8/8 (100%)



Question-level scores; N/A entries excluded from each metric's denominator.

Figure 7.2: Answer, citation, and no-evidence/security results. The denominators are 32, 29, and 5 respectively; citation judgements are at question level.

The notes describe strong metadata coverage across the eight cases, including agreement and invoice identifiers, parties, dates, the quarterly reporting period, and the gateway's product and revision. This is qualitative evidence of useful document descriptions. It does not support numerical claims about metadata precision, recall, or title quality.

7.4.4 Technical PDF classification and selective OCR

All three control PDFs receive the expected technical document label (Table 7.7). The recorded page composition agrees with their construction: three native-text pages in document 09, three scanned pages in document 10, and two native-text pages with one scanned page in document 11. The reported M11 score is three document-level passes, rather than a separately measured page-level confusion matrix.

Table 7.7: M11 technical classification of the three equivalent control PDFs.

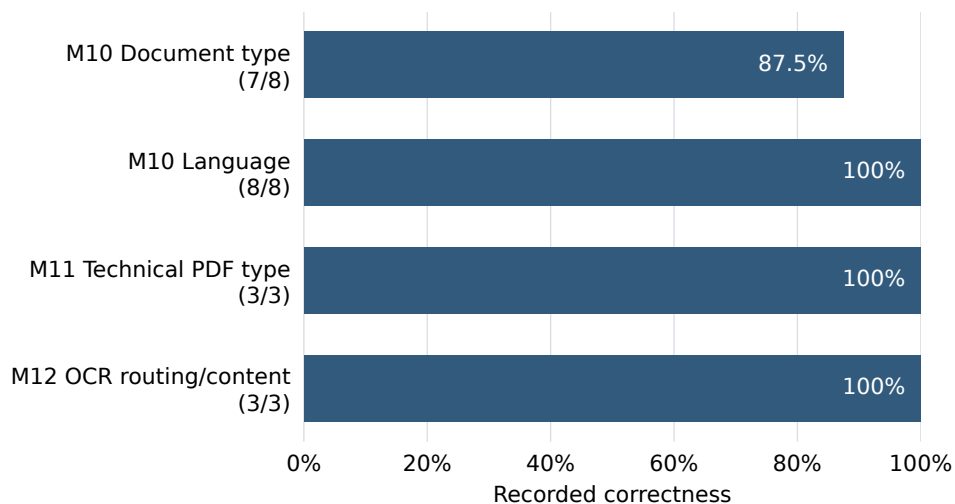
Control document	Expected / actual type	Result
09: text-based PDF	TextBased / TextBased	Pass
10: scanned PDF	Scanned / Scanned	Pass
11: mixed PDF	Mixed / Mixed	Pass
Overall	Document-level pass rate	3/3 (100%)

Table 7.8: M12 selective OCR observations. The workbook records a combined routing/content pass for each control.

Control	OCR status	OCR pages	M12 result
09: text-based	Not required	None	Pass
10: scanned	Ready	1, 2, 3	Pass
11: mixed	Ready	2 only	Pass
Overall	—	—	3/3 (100%)

Table 7.8 shows that OCR was avoided for the native control, completed for all three scanned pages, and limited to page 2 of the mixed control. The latter retained native pages 1 and 3. These observations support selective routing in the tested layouts. M12’s three passes include the native control, so 3/3 must not be read as three successful transcription tests.

The final Q13 run and Q29–Q32 obtain correct answers with first-relevant evidence at rank 1. The recovered facts include the site code RAVEN-7319, the monthly maintenance window, the 15-minute acknowledgement period, and escalation reference ESC-441-A. In Q31, the answer correctly states that the memo does not specify the action to take after an interruption continues beyond 04:15; the notes also record mention of a separate Priority 1 procedure. Its recorded pass concerns preserving that limitation rather than inventing the missing instruction.



Document-level scores; M12 does not measure character-level OCR accuracy.

Figure 7.3: Document-processing results with their separate denominators: M10 type and language use eight primary documents; M11 and M12 use three control PDFs. M12 is a recorded combined routing/content pass rate, not character-level OCR accuracy.

Figure 7.3 brings together the available processing measures while preserving the distinction between semantic labels, technical classification, and the combined OCR assessment. No aggregate across these tasks is calculated.

7.5 Discussion

7.5.1 Ranking improvements and answer completeness

The observed hybrid advantage is modest and specific: two additional questions have relevant evidence in first position, with no change to Top-3 or Top-8 coverage. Evidence within the first three positions makes it available early in the candidate ordering, although the answer generator must still select and interpret it correctly. Q04 concerns a conceptual RAG explanation, while Q11 contains the exact BlueGrid agreement identifier. The workbook attributes their promotion to hybrid retrieval and, for Q11, notes metadata involvement. These observations are compatible with complementary retrieval signals helping different query forms, but there is no isolated lexical or metadata ablation from which to assign their individual contributions.

The final stage neither improves nor worsens the reported retrieval metrics relative to hybrid retrieval. This sample therefore provides no measured aggregate gain from reranking. It also does

not establish that reranking is ineffective in larger or more ambiguous collections, particularly because application/fallback status and repeated runs are not documented.

Q28 illustrates why first-position ranking and answer correctness require separate evaluation. The evidence about service credits preserving termination rights remains at rank 2 in every ordering. Nevertheless, the answer correctly uses the relevant **S2** evidence. This is a Top-1 miss with a correct answer, rather than a complete retrieval or generation failure.

Q21 is the principal observed answer failure. It asks which documents mention Meridian Retail and what role the organization has in each. The expected set contains the NovaTel agreement and invoice, where it is the customer; the BlueGrid agreement, where it is the client; and the quarterly report, which it issues. The response identifies several roles correctly but also includes a document where Meridian Retail is absent, notably the tax-residency policy. Both answer and citation correctness are therefore marked No. Partial correctness does not compensate for the unsupported inclusion in an otherwise apparently comprehensive synthesis. This case shows that the presence of citations alone does not ensure correct evidence selection or semantic support.

This contrasts with Q19 and Q20, where focused comparisons correctly combine the two required sources. The pattern suggests that broad entity-based enumeration is a more demanding task than a comparison with explicitly named documents. The workbook establishes over-broad aggregation in Q21 but does not expose enough detail to identify whether the decisive error arose during retrieval, context selection, or answer generation. A future evaluation of such questions would need to score both coverage of the required source set and unsupported inclusions.

7.5.2 Processing and safety interpretation

The M10 mismatch shows the practical importance of the controlled-label definition. **Manual** describes the gateway document plausibly, yet differs from the required **TechnicalDocument** label. Retaining it as an error avoids revising the gold standard after observing the prediction. Correct language labels and useful metadata notes are encouraging within this corpus, but do not establish broad multilingual or entity-extraction performance.

The PDF controls demonstrate the intended distinction between native, scanned, and mixed content, including selective OCR of a single scanned page. The strongest direct evidence concerns routing and preservation of native pages. The equivalent control sources and absence of a transcription score prevent a claim that OCR is perfectly accurate. Similarly, the five successful no-evidence/security responses show appropriate abstention and handling of the tested instructions, but do not quantify robustness against a wider attack distribution.

7.5.3 Threats to validity

Internal validity. Gold expectations and pass/fail judgements are manually recorded, and no independent scorer or inter-rater agreement is documented. Binary question-level scoring can conceal differences in partial correctness. Case notes make the principal failure reviewable, but the available records do not support causal attribution to individual retrieval or generation components.

External validity. The evaluation uses 11 synthetic PDFs, of which three repeat equivalent control content. The eight primary documents cover a limited range of domains and only English and Romanian primary-language labels. Clear synthetic identifiers and controlled content may be easier than noisy, long, or heterogeneous production collections. The experiment does not establish performance for DOCX input, ImageBased or Unknown PDF cases, extreme scans, classification threshold boundaries, or the document-understanding sampling limit. Automated tests cover some related implementation risks, but are separate evidence from these application-level results.

Construct validity. First-relevant-evidence hit rate measures access to at least one supporting result, not retrieval of all required evidence. Citation judgements are per question, not an independent claim-level accuracy measurement. M12 combines routing/content observations and includes a case requiring no OCR; it is not a transcription-accuracy estimate. Metadata notes have no precision/recall denominator. These distinctions constrain the meaning of the high percentages.

Reproducibility and scope. Uploads were staged, Q13 was retested, and there is no complete frozen configuration or external model-version record. No repeated trials quantify model variability. The workbook also contains no latency, provider-usage, production-load, historical-citation-snapshot, empty-workspace, or cross-user experimental measurements. The results therefore support correct operation on most recorded cases, with a measurable hybrid Top-1 improvement and a documented failure in broad multi-document synthesis; they do not establish production-scale performance or comprehensive security.

8 | Conclusions and Future Work

8.1 Conclusions

This diploma project addressed the practical problem of finding and verifying information across an owned collection of heterogeneous documents. The resulting application accepts PDF and DOCX files, derives a searchable representation, retrieves evidence for a natural-language question, and produces an answer linked to inspectable sources. Its principal outcome is not an isolated model call, but a connected path from uploaded bytes to a persistent, authorized citation.

On the document side, the implementation validates type, signature, size, filename, and workspace ownership before storing an upload under a generated local name. A process-local background worker performs deterministic technical PDF analysis before optical character recognition (OCR). Native PDF pages and structured DOCX content are extracted in source order, while only pages classified as **Scanned** are routed through bounded PDFium rendering and local Tesseract recognition. Raw extraction remains available for inspection, and conservative normalization derives a cleaner representation without assuming that every repeated-looking line is disposable.

Document Understanding adds a separately managed enrichment stage. It uses bounded representative input to produce controlled document type, language, title, subject, and generic metadata. Strict structured output, allowlisted categories, local validation, content hashing, model and prompt identity, and atomic replacement restrict what model output can enter persistence. A failure at this stage does not prevent otherwise valid normalized text from being chunked and embedded.

The chunking design is a central engineering result. Ordered normalized sections become provenance-carrying units; oversized units are divided first at conservative sentence opportunities and then, only when necessary, at token-safe boundaries. Base chunks are accumulated toward a 700-token target, remain below the 900-token hard maximum, and aim for a 100-token useful minimum. A bounded suffix of at most 100 tokens from the immediately preceding base chunk provides continuity without recursively expanding overlap. Page ranges, source-section ranges, and headings are derived from the actual included units. The result is deterministic, multilingual-safe at string boundaries, and suitable for both embedding and citation display.

On the question side, one transient query embedding supports owner- and workspace-filtered cosine retrieval over current chunk embeddings. PostgreSQL full-text search supplies a complementary lexical channel for exact language, identifiers, and amounts. Validated document information contributes a weaker metadata signal, but only to documents already represented by vector or lexical chunk evidence. Weighted Reciprocal Rank Fusion combines ordinal positions without adding incompatible raw scores. The bounded reranker then attempts to improve the final order among the strongest hybrid candidates. It cannot create evidence, and invalid output, timeout, configuration failure, or provider failure retains the deterministic hybrid order.

Final chunks are serialized within a separate document-context token budget. The backend

assigns source identifiers, the answer model is instructed to rely only on the delimited untrusted evidence, and locally accepted citations must belong to the context map. Conversation history is persistent and bounded, but it is explicitly non-authoritative and does not replace retrieval for the current question. Bounded source snapshots preserve the historical meaning of a citation even if its live document is later removed.

The completed implementation is supported by 345 passing backend tests, with no failures or skipped cases in the single verified full run. The tests cover successful paths, ownership isolation, deterministic algorithms, malformed model output, prompt-like document content, provider failures, and fallback behavior. This is evidence that specified software behavior is implemented; it complements the completed experimental evaluation reported in Chapter 7. That evaluation records a hybrid Top-1 gain within the tested corpus, no further aggregate gain from the final ordering, and an answer/citation failure in broad multi-document synthesis.

8.2 Personal Contribution and Lessons Learned

The personal contribution consists of designing and implementing how established techniques cooperate inside one application. The work includes the server domain and persistence model, authenticated workspace boundary, ingestion orchestration, extraction and normalization, technical analysis, selective OCR, Document Understanding validation, token-aware chunking, embedding currentness, semantic and lexical database retrieval, metadata-aware weighted RRF, bounded reranking, grounded answering, authoritative source mapping, conversation persistence, the React user experience, diagnostic views, and automated tests.

Several design lessons emerged from this integration. First, provenance must be created at extraction time rather than reconstructed after an answer is generated. A page number or heading that is discarded before chunking cannot later be recovered reliably for a citation. Second, deterministic code should retain authority around probabilistic services. Model output is most useful when its purpose is narrow, its input and output are bounded, and local code validates identities and controlled fields.

Third, failure states should reflect independent concerns. A document can be searchable even when optional understanding fails; OCR can be partial while useful native and recognized pages remain; and hybrid retrieval can continue when reranking is unavailable. At the same time, failure isolation must not hide lost evidence. A document with scan-only pages and no successful recognition cannot honestly become ready merely because an optional stage is defined as non-fatal.

Fourth, a conversational surface benefits from exposing a limited diagnostic path. Most users need a question box, an answer, and sources. Development and evaluation require page types, OCR provenance, normalization views, chunk locations, channel ranks, fused rank, reranked rank, and fallback state. Keeping those details available but secondary allows the same application to remain usable and explainable.

8.3 Current Limitations

The implementation has practical limitations that define the scope of its conclusions:

- Embeddings, Document Understanding, model reranking, and answer generation depend on the configured external model provider. Local OCR does not make the complete AI pipeline local, and provider availability, latency, model behavior, and usage cost affect these stages.
- The reranker adds a second model request for applicable searches. Its candidate, token,

and timeout limits control cost and latency, but can exclude useful evidence from joint comparison. Its relevance grades are ordinal diagnostics rather than calibrated probabilities.

- Technical PDF classification is based on versioned deterministic thresholds for text and raster structure. Unusual layouts, hidden text layers, vector-only graphics, or images that do not meet the scan threshold can be classified imperfectly and therefore routed imperfectly.
- OCR quality remains sensitive to resolution, skew, compression, typography, language data, and page layout. The current route does not implement image preprocessing such as deskewing, denoising, or adaptive contrast enhancement.
- DOCX extraction covers body paragraphs, Heading 1–3 styles, and tables in order. It does not reconstruct printed pagination or explicitly extract every possible Office Open XML feature such as comments, footnotes, headers, footers, or drawing text.
- Retrieval uses bounded vector, lexical, metadata, fusion, reranking, TopK, and context pools. A relevant passage outside those bounds cannot support the answer. The current schema does not define an HNSW or IVFFlat approximate vector index, so larger-scale vector-search performance has not been established.
- The background queue is bounded, process-local, and non-durable. A process restart loses queued identifiers, and a full queue may require a document to be scheduled again. Local file storage likewise assumes one application environment rather than a distributed deployment.
- One user owns each workspace; sharing and role-based collaboration are not implemented. The repository also does not define a production deployment, distributed monitoring, or durable object-storage architecture.
- Backend behavior has extensive automated coverage, but the React client has no automated test suite and accessibility has not undergone a formal compliance audit or assistive-technology study.
- The completed evaluation is limited to a small synthetic corpus and manually scored cases. It does not establish general retrieval or security robustness, character-level OCR accuracy, or production-scale performance; latency and provider usage were not measured.

These limitations do not negate the completed engineering flow. They identify where a local, feature-complete bachelor project ends and where deployment hardening or broader empirical study would begin.

8.4 Future Work

Future evaluation should extend the completed study in Chapter 7 with longer documents, noisier scans, additional layouts, multilingual queries, ambiguous cases, and independently labelled relevance judgements. Measurements should guide changes to chunking, candidate pools, fusion weights, reranking prompts, and context limits rather than adjusting them from isolated demonstrations.

For ingestion reliability, the process-local channel could be replaced by a durable job queue with retry policy, visibility, concurrency control, and recovery after restart. Uploaded content could move from one local directory to managed object storage, with checksum verification and retention policy. Production deployment would additionally require environment-specific secret management, transport and storage security, observability, rate limiting, backup, and operational testing.

OCR could be extended with controlled image preprocessing, orientation detection, deskewing, denoising, and additional language packs. Any preprocessing route should be evaluated separately because a transformation that improves one scan can damage another. Technical classifica-

tion could incorporate further structural signals while retaining a versioned, explainable decision record.

At larger collection sizes, PostgreSQL vector search could be evaluated with an approximate nearest-neighbour index such as HNSW or IVFFlat. The choice should be based on measured latency, memory, ingestion cost, and recall under the actual workspace filters. Retrieval could also support explicit user-selected document scope or date/type filters, while preserving the present default in which metadata is a soft signal rather than a hidden exclusion rule.

Alternative or locally hosted embedding, understanding, reranking, and answer models could reduce provider dependency or support deployments with stricter data constraints. Such substitution is not merely a configuration exercise: dimensions, tokenization, structured-output reliability, multilingual quality, context size, latency, and evaluation results would all need to be revalidated.

Finally, the client would benefit from automated component and end-to-end tests, a formal keyboard and screen-reader review, and deployment-scale usability evaluation. These steps would extend the current accessibility considerations into measured evidence without overstating the present implementation.

8.5 Final Perspective

The project demonstrates how a document assistant can be built as an accountable engineering pipeline. Its usefulness depends on the connections between stages: page analysis determines OCR routing; extraction and normalization determine chunks; chunks and metadata determine retrieval; retrieval determines context; and backend source mappings determine which citations the interface can trust. Treating these connections as first-class design concerns distinguishes the system from a generic file-upload chat interface.

The application is feature-complete for the stated project scope, with quantitative results and their limitations reported in Chapter 7. These observations support the technical account within the evaluated corpus while leaving broader empirical claims open.